Zinnia: Expressive, Efficient Zero-Knowledge Framework for General-Purpose Data Analytics

Zhantong Xue
Hong Kong University of Science and Technology
Hong Kong, China
zxueai@cse.ust.hk

Pingchuan Ma*
Hong Kong University of Science and Technology
Hong Kong, China
CipherInsight Limited
Hong Kong, China
pmaab@cse.ust.hk

Zhaoyu Wang
Hong Kong University of Science and Technology
Hong Kong, China
zwangjz@cse.ust.hk

Shuai Wang*
Hong Kong University of Science and Technology
Hong Kong, China
CipherInsight Limited
Hong Kong, China
shuaiw@cse.ust.hk

Abstract

Data analytics is a powerful tool for uncovering patterns and generating insights. However, once a claim is made based on data analysis, its audience must either trust the analyst or re-execute the analysis (often on private or proprietary data) to verify its correctness. This reliance raises significant concerns about transparency and trust in the analytics process. Zero-Knowledge Proofs (ZKPs), a cryptographic technique, offer a principled solution by enabling analysts to produce proofs of correctness without revealing the underlying data. This paradigm, known as verifiable computation, allows any verifier to check the validity of the analysis result solely from the proof.

In this paper, we introduce Zinnia, a expressive and efficient ZKP framework designed for general-purpose data analytics. Zinnia provides a high-level domain-specific language (DSL) for encoding analytics workflows and a symbolic execution engine that reasons about the programs and compiles them into optimized ZKP circuits. Together, these components support rich language features such as data-dependent control flow (e.g., dynamic loops, recursion, early exits), real-valued arithmetic, non-linear functions and multi-dimensional array manipulations. We evaluate Zinnia's usability through real-world case studies and a user study, and benchmark its performance across diverse analytics tasks. Zinnia achieves up to 5.8× speedup over zkVMs and produces ZKP circuits that are 19.3% smaller than those generated by existing zero-knowledge programming languages.

1 Introduction

Data analytics underpins scientific discovery, operational decision making, and public-policy formulation. Modern analysts wield an extensive arsenal of ecosystems to ingest, transform, aggregate, model, and visualize data at every scale. The data analysts make claims about the results of their computations, but provide no means to verify the correctness of these claims. A straightforward solution is to release the raw data where the stakeholders can re-execute the computations and verify the results. However, this approach is often infeasible when the data is private or proprietary. As a result, transparency and credibility of data analytics are increasingly challenged.

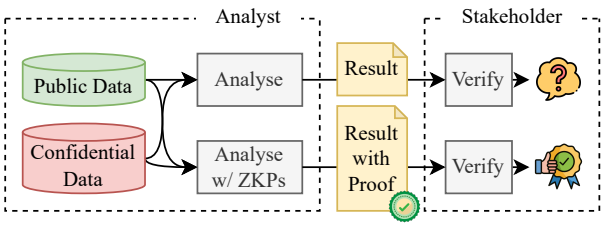

Figure 1: Conceptual data-analytics scenario with ZKPs. Analysts can convince stakeholders of the validity of their results without revealing sensitive data.

ZKPs (Zero-Knowledge Proofs) [10] provide a principled solution to this problem. As shown in Fig. 1, ZKPs allow a *prover* to convince a *verifier* that a *computational statement* is evaluated correctly on secret inputs, while revealing nothing beyond the claim itself. As illustrated in Fig. 2, the mechanical workflow of ZKPs consists of four main steps: (1) the prover expresses their computational statement into a static arithmetic circuit, (2) the prover computes the output of the circuit together with an auxiliary witness, (3) the prover emits a succinct proof of correct execution, and (4) the verifier checks the proof against the public inputs and the arithmetic circuit. This paradigm enables verifiable computation where the prover can convince the verifier of the correctness of their computations without exposing private data. With the success of recent SNARK systems [9, 11, 23], the proof is non-interactive, its size is small enough to be practical for real-world applications, and the verification is often in constant time.

Technical Challenges. Despite the promise of ZKPs for verifiable computation, applying them to general-purpose data analytics remains challenging. As illustrated in Fig. 2, the arithmetic-circuit model underlying most ZKP systems, such as SNARKs, imposes rigid structural constraints. Restricted to static-topology circuits,

*Corresponding author

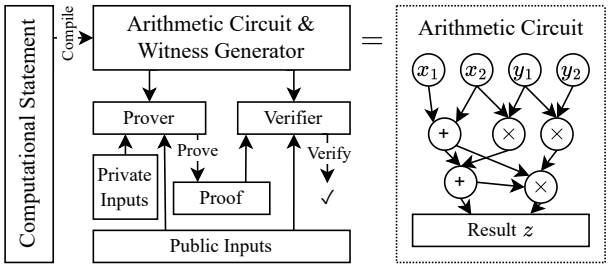

Figure 2: Standard ZKP workflow. The arithmetic circuits are fixed and input-independent.

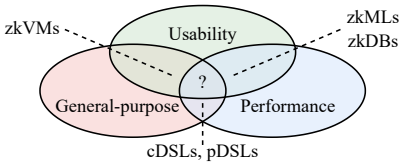

Figure 3: Comparison of existing solutions.

they are poorly suited to the rich control flow and data structures in real-world data analytics workloads. In particular, data analytics tasks often require data-dependent control flow (e.g., dynamic loops, recursion, early exits), real-number arithmetic, and flexible indexing over multidimensional arrays, which are inherently difficult to express in low-level arithmetic circuits. These semantic gaps hinder the direct application of ZKPs to practical analytics scenarios without significant abstraction or tooling support.

Existing Solutions. To bridge this gap, the ZKP ecosystem has introduced a range of solutions as shown in Fig. 3. Domain-specific systems, such as zkML frameworks [12, 32, 47, 49] and zkDB engines [24, 31, 50], focus on specialized workloads like machine learning or SQL queries. They offer high efficiency at the cost of generality. At the programming level, constraint domain-specific languages (cDSLs) such as Circom [7] and Halo2 [14] provide abstractions over arithmetic circuits but remain low-level, requiring manual gate-level construction. Program DSLs (pDSLs) like Noir [40] and Leo [13] offer higher-level syntax and achieve comparable performance to cDSLs, but still restrict expressiveness with fixed-length loops, unrolled conditionals, and limited UDF support. In addition, their supports to real-valued arithmetic, dynamic arrays or other important data analytics primitives are either limited or non-existent. To improve flexibility, zkVMs [30] such as SP1 [28], RISC0 [52] and Cairo [48] emulate general-purpose programs at the CPU instruction level, which provides compatibility with arbitrary Rust code. However, this approach introduces significant performance overhead and is primarily designed to lightweight tasks (e.g., blockchain transaction verification) instead of compute-intensive analytics pipelines.

Our Solution. To overcome the limitations, we present Zinnia, a zero-knowledge framework for general-purpose data analytics. Zinnia combines a DSL with native support for data-dependent control flow and a symbolic execution engine that compiles programs into optimized ZKP circuits. Unlike traditional execution, which runs programs on concrete inputs, the symbolic execution engine interprets programs over symbolic values, which allows it to analyze all possible execution paths that depend on input data. This analysis allows Zinnia to generate a static arithmetic circuit that captures the full semantics of the program and is independent of any specific input. On the basis of the DSL and the symbolic execution engine, Zinnia implements essential data analytics features including real-valued arithmetic, non-linear functions and multidimensional array manipulations. Through case studies and a user study, we show that Zinnia offers a convenient and expressive programming interface for developers working with ZKPs. Across a variety of data analytics workloads, our benchmarks indicate that Zinnia achieves at least a 5.8× speedup over emulation-based zkVMs and produces arithmetic circuits that are at least 19.3% smaller than those generated by existing DSLs on two different protocols. We summarize our contributions as follows:

- **DSL and Compiler.** We design a new DSL with the native support for data-dependent control flow. We design a symbolic execution engine to compile the DSL programs into optimized ZK-friendly intermediate representation (IR) traces, which are then synthesized to efficient ZKP arithmetic circuits.
- **Data Analytics Module.** On top of the programming interface, we provide a suite of data analytics features, including real-valued arithmetic, non-linear functions and multidimensional array manipulations, to effectively enable real-world data analytics workloads.
- **Evaluation.** We evaluate Zinnia on a variety of data analytics workloads, including real-world case studies and a user study. The results show that Zinnia is both expressive and efficient with reduced development effort and improved performance.

2 Background

2.1 Zero-Knowledge Proofs and SNARKs

Let $\mathcal{R}$ be a relation on fields defined by a predicate $\alpha(s, w)$ that runs in time polynomial in the lengths of s and w, with the convention that $(s, w) \in \mathcal{R}$ exactly when $\alpha(s, w) = 1$, with α serving as its polynomial-time verifier. The language $\mathcal{L} = \{ s : \exists w \text{ such that } \alpha(s, w) = 1\}$ is therefore in NP. For any statement $s \in \mathcal{L}$, a Zero-Knowledge Proof is a protocol where a prover who knows a witness w for which $\alpha(s, w) = 1$ convincingly demonstrates to a verifier that such a w exists, while revealing nothing beyond the fact that $s \in \mathcal{L}$. Throughout the interaction the verifier's only inputs are the statement s and the description of the predicate α; the witness w remains perfectly hidden. The protocol consists of three algorithms executed on common public parameters:

- **Setup.** On input the security parameter λ and the statement s, $\mathsf{Setup}(\lambda, s)$ outputs a proving key pk and a verification key vk.
- **Proving.** Using pk and the witness w, the prover runs $\mathsf{Prove}(\mathsf{pk}, w)$ to generate a succinct proof π.
- **Verification.** The verifier applies $\mathsf{Verify}(\mathsf{vk}, \pi)$, which runs in time polylogarithmic in $|s|$. If the output is 1, the verifier is convinced that $\alpha(s, w) = 1$ and thus that s is valid.

A ZKP system for the relation $\mathcal{R}$ must satisfy three properties: *Completeness* means that if $(s, w) \in \mathcal{R}$, then an honest prover who knows w convinces the verifier to accept except with negligible

probability. *Soundness* means that if no w satisfies $\alpha(s, w) = 1$, then even a malicious prover cannot make the verifier accept except with negligible probability. *Zero-knowledge* means that whatever the verifier sees during the protocol can be simulated without access to w, so it learns nothing beyond the fact that $(s, w) \in \mathcal{R}$.

SNARKs are a family of ZKPs with small proof sizes and alleviate the need of interactions between the prover and verifier, which makes them particularly suitable for practical applications. SNARKs require expressing the statement s as a static arithmetic circuit c under certain arithmetic systems (e.g., R1CS, PLONK). The private witness w is then an assignment to c's inputs and internal wires satisfying $c(w) = 1$ if and only if s is true. Since the circuit c is fixed at compile time, encoding dynamic control flow, complex data structures or other high-level constructs requires substantial task-specific circuit engineering effort.

2.2 Existing ZKP Solutions

We review existing ZKP solutions and discuss their limitations in the context of data analytics. We categorize them into three main classes: *declarative libraries*, *program-based DSLs (pDSLs)*, and *zero-knowledge virtual machines (zkVMs)* and summarize their key features in Table 1.

Table 1: Feature comparison of ZKP solutions for data analytics. (✓= fully supported, ×= not supported, ○= partially supported.)

			cDSL	pDSL	zkVM	Zinnia
Performance			✓	✓	×	✓
Expressiveness	Constructs	Dynamic Loop	×	×	✓	✓
		Recursion	×	×	✓	✓
		Func Return	×	○	✓	✓
		Index & Slice	×	○	✓	✓
	Data Ops.	Real number	×	×	✓	✓
		Non-linear Op	×	×	✓	✓
		Multidim-Arr	×	○	✓	✓
		Multidim-Op	×	×	○	✓

Declarative Libraries and *Constraint-based DSLs* such as Circom [7], Halo2 [14], and LibSNARK [45], expose APIs for direct arithmetic circuit construction, representing the most low-level programming model. While offering maximum flexibility in circuit design, they inherently burden developers with arduous, error-prone, manual circuit engineering. Crucially for data analytics, these tools provide no native support for abstracting complex data structures or dynamic computations, forcing users to manually decompose high-level data operations into granular arithmetic gates.

Program-based DSLs (pDSLs) like ZoKrates [17], Leo [13], and Noir [40] offer a more expressive, higher-level programming model. They allow developers to write imperative programs with a limited set of control-flow structures that are automatically compiled into arithmetic circuits, abstracting many low-level details of constraint management. However, they still sacrifice expressiveness and are restricted to conditional branches and static, fixed-length loops. This limits their applicability for data analytics, which often requires dynamic iteration based on data-dependent conditions, recursive function calls, and arbitrary array indexing and slicing. These restrictions make them difficult to support complex tasks such as dynamic filtering, aggregation, or graph traversals.

Zero-knowledge Virtual Machines (zkVMs) such as SP1 [28], Cairo [48], and RISC0 [52], represent a significant improvement on usability. They allow developers to write programs in high-level languages (e.g., Rust) and execute them within a virtual machine that emulates a CPU instruction set. The execution trace is then compiled into a ZKP circuit that verifies the correctness of each state transition. However, the flexibility of zkVMs comes at a cost. The instruction-level emulation introduces substantial overhead, as the entire program execution trace must be captured and compiled into a circuit. This results in large proof sizes and long proving times (typically in the order of magnitude slower than cDSLs/pDSLs). As a result, zkVMs are often impractical for data analytics workloads, which typically involve large datasets and complex operations that require efficient computation and verification.

Motivated by the current landscape, our work introduces Zinnia, a framework designed to overcome two key limitations in general-purpose ZKP data analytics: ① the rigidity of arithmetic circuits, which enforces static, input-independent programs and precludes dynamic, data-dependent control flow; and ② the lack of systematic support for the rich data manipulation and arithmetic operations that are prevalent in real-world analytics workloads. We provide an overview of Zinnia's design in Sec. 3.

3 Research Overview

3.1 Architecture and Workflow

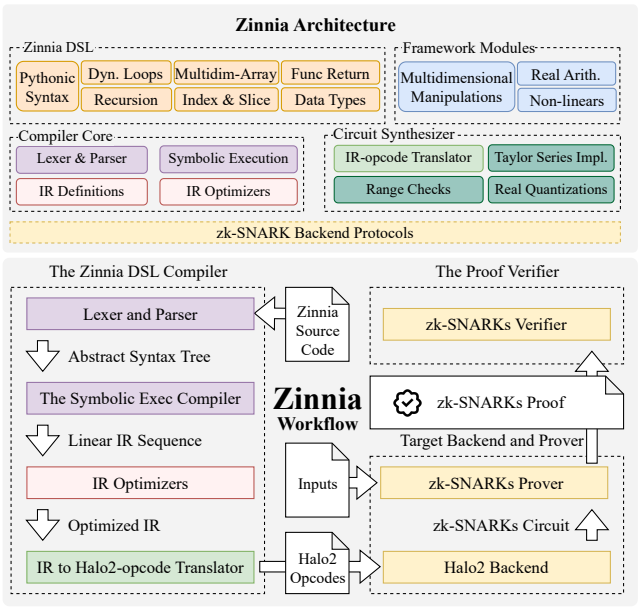

Figure 4: Zinnia's architecture and workflow overview.

We present the architecture and workflow of our system in Fig. 4.

DSL and Compiler (Sec. 4). We first provide a DSL and its compiler, where data analysts can express their computations in a high-level and data-centric manner. In essence, our DSL supports a collection of modern program constructs that are instrumental for real-world data analytics, including dynamic loops, recursion, early returns, and dynamic indexing. Since these constructs involve data-dependent control flow, they cannot be directly translated into static circuits. To address this, our compiler performs symbolic execution to reason about the program's control flow and data dependencies. It explores all possible control paths, tracks data dependencies, detects and prunes unreachable code paths, folds constants, and emits a linearized intermediate representation (IR) trace. This ensures that all data-dependent behavior is statically lowered to a ZKP-friendly and fixed sequence of opcodes, which will be used for subsequent circuit synthesis. To further improve performance, we apply a series of compiler optimizations to the IR trace, including dead-code elimination, common-subexpression elimination and pattern-matching rewrites.

Data Analytics Module (Sec. 5). It is worth noting that ZKP systems operate on integers within a large prime field, which is not directly suitable for data analytics tasks that often require real number support and complex data structures. Thus, DSL and compiler alone are not sufficient for enabling real-world data analytics workflows. To this end, we implement a set of data analytics primitives that are commonly used in data processing and machine learning workflows. These primitives include real-valued arithmetic, non-linear functions and multidimensional array manipulations, which abstract away the underlying complex arithmetics on integers in a prime field.

Circuit Synthesizer (Sec. 6). With the above modules, developers can encode their data analytics programs in our DSL, which are then compiled into a sequence of IR opcodes. The next step is to synthesize these opcodes into a static arithmetic circuit that can be used for ZKP systems. It is worth noting that Zinnia is agnostic to the underlying proving backend, and can be used with any SNARK protocol that supports arithmetic circuits. Specifically, we implement the circuit synthesizer that takes the IR opcodes and emits API calls of Halo2-lib [4] or Noir [40], depending on the chosen proving backend. These API calls instantiate pre-defined gates and constraints in the underlying arithmetic systems (i.e., PLONK for Halo2-lib and ULTRAHONK for Noir). In this way, we separate the IR design from the circuit emission, keeping the lowering pipeline modular and making it straightforward to swap in alternative proving backends and SNARK protocols.

Workflow. Fig. 4 illustrates how a data analytics program written in our DSL is processed by Zinnia. The first step is to compile the program (as well as the data analytics primitives) into a sequence of IR opcodes, which are then passed to the circuit synthesizer. The circuit synthesizer emits a static arithmetic circuit that captures the semantics of the program. Next, the data analyst (prover) provides private inputs to the circuit, which are then committed to using a hiding commitment scheme. The proving backend consumes both the arithmetic circuit and the confidential inputs to produce a succinct proof of correct execution. Finally, the prover distributes the proof, with which any verifier can validate the computation without learning anything about the private data.

3.2 Threat Model

Zinnia targets analytics workflows in which an analyst (prover) holds private data and wants to make claims on the results of computations performed on that data. The verifier, however, is an external party that does not have access to the private inputs and wants to verify the correctness of the claims made by the prover. Thus, in the threat model, on one hand, the prover may attempt to cheat by fabricating a proof that does not correspond to the actual computation. On the other hand, the verifier may attempt to learn more about the private data than what is allowed by the prover. Zinnia inherits the standard security assumptions of its underlying proving backend. Upon these assumptions, Zinnia guarantees that the prover cannot produce a fraudulent proof that passes verification, and the verifier cannot learn anything about the private inputs beyond what is revealed through the public outputs of the computation.

4 Zinnia DSL and Compiler

4.1 DSL and Programming Model

⟨Prog⟩	P	::=	$\epsilon \mid s; P$
⟨Stmt⟩	s	::=	$s_1; s_2 \mid e \mid \texttt{if}\ e : s_1\ \texttt{else} : s_2 \mid \texttt{if}\ e : s$
		|	$\texttt{for}\ id\ \texttt{in}\ e : s \mid \texttt{while}\ e : s$
		|	$\texttt{return}\ e \mid \texttt{assert}\ e \mid id \leftarrow e \mid e_1[e_2] \leftarrow e_3$
		|	$s^{\text{in}} \mid s^{\text{func}} \mid \texttt{pass} \mid \texttt{continue} \mid \texttt{break}$
⟨Func⟩	s^{func}	::=	$id^{\text{func}}(id_1 : t_1, \ldots, id_N : t_N) \rightarrow t_r : s$
⟨Input⟩	s^{in}	::=	$\texttt{input\_public}(id, t) \mid \texttt{input\_private}(id, t)$
⟨Lit⟩	c	::=	⟨Int⟩ | ⟨Float⟩ | True | False | None
⟨Type⟩	t	::=	$\texttt{Int} \mid \texttt{Bool} \mid \texttt{Float} \mid \texttt{Array}[t; n]$
		|	$\texttt{List}[t_1, t_2, \ldots, t_N] \mid \texttt{Tuple}[t_1, t_2, \ldots, t_N]$
⟨Expr⟩	e	::=	$id \mid c \mid t \mid e_1 \otimes e_2 \mid \odot e \mid e_1[e_2]$
		|	$[e_1, \ldots, e_N] \mid (e_1, \ldots, e_N)$
		|	$id(e_1, \ldots, e_N) \mid op(e_1, \ldots, e_N)$
⟨Bin-Op⟩	$\otimes$	::=	+ | − | ∗ | / | % | ∗∗ | @ | // | and | or
⟨U-Op⟩	$\odot$	::=	+ | − | not

Figure 5: The Syntax of Zinnia DSL.

As a key design consideration, we aim to provide a DSL that is intuitive and easy to use for data analysts. To this end, we follow the design principles in Python where the programs written in our DSL are also valid Python programs. We present the syntax of our DSL in Fig. 5, which is a subset of Python. The DSL supports a collection of modern program constructs (e.g., dynamic loops, early returns, recursion). In the meantime, the developers annotate their data with type hints, which are used for deciding the circuit layout, data representation and declaring public/private inputs. The DSL supports common elementary data types, including integers, floats, booleans, lists, tuples, and multidimensional arrays. On top of these elementary constructs, it offers basic unary and binary operators (i.e., U-Op and Bin-Op in Fig. 5). As will be introduced shortly, additional higher-level operations are provided through our data analytics primitives.

To understand our DSL and programming model, we present three examples in Fig. 6. Besides, we also provide additional examples in the appendix, which would cover the full language features.

```python
@zk_circuit
def data_binning(data: NDArray[float, 2, 5], result: NDArray[float, 2, 1]):
    new_data = data[:, ::-1]
    bin_data_mean = new_data[:, :(data.shape[1] // 3) * 3].reshape(
        (data.shape[0], 1, 3)).sum(axis=-1) / 3
    assert result == bin_data_mean
```

```python
@zk_chip
def is_prime(number: int) -> bool:
    assert 0 <= number <= 10000
    if number < 2:
        return False
    for i in range(2, 101):
        if i * i > number:
            return True
        if number % i == 0:
            return False
    return True
```

```python
@zk_circuit
def calc_entropy(
    X: NDArray[float, 10],
    ans: float
):
    assert sum(X) == 1.0
    assert ans == np.sum(
        -X @ np.log2(X.T))
```

Figure 6: Example Programs in ZINNIA DSL.

EXAMPLE 1. `data_binning` *partitions a 2D time series into fixed-size bins (size 3) and computes the average of each bin. This example showcases multidimensional array support and operations such as indexing, slicing, reshaping, and axis-wise aggregation. The* `@zk_circuit` *decorator signals that the function should be compiled into an arithmetic circuit. All inputs must have statically known sizes at compile time, as required by arithmetic circuits' design.*

EXAMPLE 2. `calc_entropy` *computes Shannon entropy, highlighting support for non-linear operations (e.g.,* `log`*) and real-numbers.*

EXAMPLE 3. `is_prime` *implements a primality test with a dynamic loop and an early* `return`*, immediately halting once compositeness is detected. While not illustrated here, ZINNIA also supports* `break` *and* `continue`*. This dynamic data-dependent control flow for loop exits, skips, and early function returns are what prior DSLs lack because of the underlying static circuit model.*

4.2 Compilation Pipeline

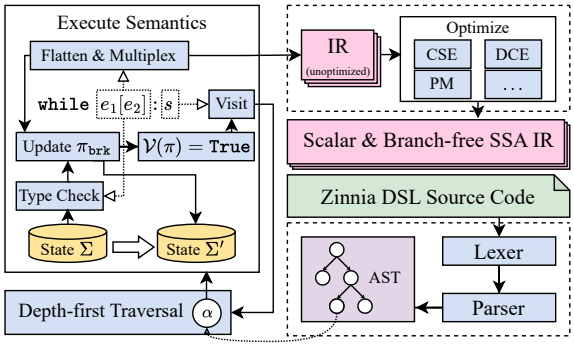

Figure 7: Overview of the ZINNIACC Compilation Pipeline.

With programs written in our DSL, the next step is to compile them into ZKP-friendly intermediate representation (IR) opcodes. To this end, we design a compiler, ZINNIACC, which leverages a symbolic execution engine to lower ZINNIA programs into the opcodes of ZINNIA's IR. These IR opcodes are then synthesized into a ZKP circuit. Fig. 7 provides an overview of how ZINNIACC translates a ZINNIA program into a sequence of IR opcodes. Starting from the source code (the green box in Fig. 7), the compiler first lexes and parses the code into an abstract syntax tree (AST). With the AST, a symbolic execution engine is invoked to traverse the AST. At its core, it maintains a symbolic state $\Sigma = (\Gamma, \Pi, \sigma)$, which tracks the program's control flow and data manipulation. The symbolic execution engine applies a set of formal semantic rules to process the AST and compile it into a sequence of IR opcodes. Through this process, control-flow constructs are handled, where loops and recursion are unrolled to statically inferred bounds, assignments and assertions are guarded by path conditions, infeasible branches are pruned, and all composite data structures are flattened into scalar expressions. To accurately reflect the data type (for subsequent circuit synthesis), the compiler also performs type inference and checking, with which we can infer the size of variables and assign suitable operations for IR generation (the upper pink box in Fig. 7). Finally, the compiler optimizes the generated IR with compilation optimization techniques (e.g., dead-code elimination, common subexpression elimination, and pattern-matching rewrites) to obtain a scalar and branch-free IR in the static single assignment form. Below, we present a running example to illustrate the compilation process.

EXAMPLE 4. *Consider the loop* `while` $e_1[e_2]$: s *where* e_1 *is an array and* e_2 *an index expression. During compilation, the array lookup is lowered to a multiplexor over each element of* e_1*:* $cond = \sum_{i=0}^{n-1}(e_2 = i) \cdot e_1[i]$*. The compiler then unrolls the loop up to a statically inferred bound (i.e. until* $cond = 0$*), executing the body at each iteration under the guard* $cond$*. After each pass,* $cond$ *and any loop-modified variables in the symbolic state* Σ *are updated. Infeasible iterations (where the guard is unsatisfiable under* π*) are pruned, yielding a flat, branch-free IR sequence that precisely mirrors the data-dependent behavior of the original loop.*

In the following sections, we elaborate on the design of symbolic state representation, the operational semantics that translate language constructs into IR sequences, and the type system that enforces correctness throughout the transformation.

4.3 Symbolic State Representation

In a typical interpreter's execution, the "state" represents the values of all variables, the call stack, and the program memory. The state evolves as the program executes, and reflects the program's control flow and data manipulation [46]. In a similar vein, we maintain a symbolic state $\Sigma = (\Gamma, \Pi, \sigma)$ during the compilation process. Here, Γ denotes the current program frame, Π is the set of accumulated IR statements, and σ is the memory store. In the following, we elaborate on each component in this state. We also provide an overview of this representation in Fig. 8.

4.3.1 Environment Γ*.* The environment Γ models the program's scope and is structured as a linked list of frames [1]. It resolves variable names and tracks the program's control flow. Each frame represents a distinct scope of the program. With a sequence of linked frames, the compiler can navigate through the program's scopes and ensure accurate variable resolution.

Frames are basic data structures designed to represent program scopes within our compiler. Each frame encapsulates a distinct

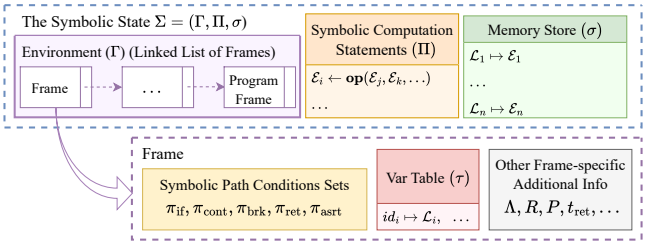

Figure 8: ZinniaCC's Symbolic State Representation.

scope and includes a variable table τ, which maps variable names to their memory locations. Additionally, every frame stores path conditions that must be satisfied for reaching the specific point represented by the frame. The parent frame of the current frame is denoted by $\hat{\Gamma}$. When resolving a variable, the compiler first searches the current frame's variable table and then recursively traverses its parent frames until the variable is found or the top-level frame is reached. We define three special types of frames with different metadata in addition to the common variable table, path conditions, and a pointer to its parent.

- *Program Frame* serves as the root-level frame for the entire program. It includes a table Λ for user-defined functions and a set P for public inputs.
- *If Frame* and *Loop Frame* store the condition expressions pertinent to their respective control flow constructs.
- *Func Frame* stores the function's return type t_{ret} and a list of return values paired with the corresponding conditions R.

Path conditions are symbolic predicates attached to execution frames that capture control-flow decisions. Specifically, π_{if} guards entry into an `if` branch; π_{brk}, π_{cont}, and π_{ret} govern loop exits, iteration skips, and early function returns, respectively, (each "skipping" subsequent code when true); and π_{asrt} enables assertions. At any program point with environment Γ, the effective condition for execution is the conjunction

$$\pi^{\Gamma} = \left(\bigwedge\nolimits_{x \in \pi^{\Gamma}_{\text{if}}} x\right) \wedge \left(\bigwedge\nolimits_{x \in \pi^{\Gamma}_{\text{brk}} \cup \pi^{\Gamma}_{\text{cont}} \cup \pi^{\Gamma}_{\text{ret}}} \neg x\right)$$

Variable Table τ, as a part of a frame, is a mapping from variable names to their memory locations. The set of all variables within environment Γ is denoted as τ^{Γ}, and the memory location of variable id is denoted as τ^{Γ}_{id}.

4.3.2 IR Statements Π. The set of IR statements, denoted Π, constitutes the direct output of the compilation process. Each statement in Π is a static single-assignment expression structured as $\mathcal{E}_i \leftarrow \mathbf{op}(\mathcal{E}_{i_1}, \mathcal{E}_{i_2}, \ldots)$, where $\mathcal{E}$ represents a symbolic expression and **op** signifies an operator (opcode) over finite fields. These statements collectively form the ultimate result produced by the compiler. We integrate Π as a component of the symbolic state to accurately reflect the accumulation of compiled results throughout the symbolic execution.

4.3.3 The Memory Store σ. The memory store σ is defined as a mapping from concrete memory addresses to symbolic expressions. It is specifically devised to model the program's memory state during symbolic execution, playing a crucial role in accurately handling operations involving array indexing and updates, ensuring that memory accesses are resolved to their corresponding symbolic values. We discuss the memory aliasing problem in the appendix.

4.4 Operational Semantics and Symbolic Execution

With the representation of the symbolic state, we now present how it evolve over the symbolic execution of a program. Technically, we formalize the evolution rules through operational semantics in Fig. 9, which define how each program construct transforms the symbolic state.

Given a symbolic state $\Sigma = (\Gamma, \Pi, \sigma)$, compilation steps for program constructs are formalized using judgments of the form

$$\Gamma, \Pi, \sigma \vdash \{e\} \rhd \Gamma', \Pi', \sigma', \langle \mathcal{L}, \mathcal{E} \rangle \tag{1}$$

This judgment indicates the compilation of a program construct $\{e\}$ starting from an initial symbolic state (Γ, Π, σ). The compilation process transforms the state to (Γ', Π', σ'), and, if a value is produced, yields an evaluation result $\langle \mathcal{L}, \mathcal{E} \rangle$. In this tuple, $\mathcal{L}$ represents the memory location of the evaluated value, and $\mathcal{E}$ its symbolic expression. Furthermore, we define a function $\mathcal{V}(\mathcal{E})$ to denote the resolution mapping a symbolic expression $\mathcal{E}$ to its concrete value (see Sec. 4.5). Given that $\mathcal{E}$ may contain free variables, $\mathcal{V}(\mathcal{E})$ yields a constant value if $\mathcal{E}$ is provably constant, or a designated "unknown" result otherwise. We define $\mathcal{T}$ (see Sec. 4.6) as the type inference function, which maps an expression to its type (see Sec. 4.6).

Due to space limit, we only present representative examples of operational semantics in Fig. 9. The remaining rules are available in the appendix. Below, we provide a brief overview of the semantics for each program construct.

- *Expressions.* Var-Load-Expr fetches a variable's address then its symbolic value. Literal-Expr allocates a fresh location for a literal and updates the store. Binary-Expr (resp. Unary-Expr) recursively evaluates operands, applies the operator, and stores the result in a new location.
- *Program Inputs.* Each input statement allocates a fresh symbol α^{t}_{id}, updates the variable table and store, and tags metadata: set P marks Input-Stmt (Public), hashed inputs additionally check the supplied hash. The compilation core is identical for Input-Stmt (Public), Input-Stmt (Private), and hashed inputs.
- *Conditional Branch.* If-Stmt evaluates its condition e_1, records it in an `IfFrame`, then evaluates the branch.
- *Loop.* Loops are statically unrolled. For For-In-Stmt, the unroll count equals the statically known length of iterable e. A `LoopFrame` stores path conditions π_{cont} and π_{brk}; π_{cont} is cleared each iteration. `while` updates π_{brk} with the evaluated condition and continues until $\mathcal{V}(\mathcal{E}_i) \wedge (\bigwedge_{x \in \pi_{\text{brk}}} x)$ is false.
- *Loop Controls.* Continue-Stmt adds the current path condition π^{Γ} to π_{cont}; Break-Stmt adds it to π_{brk}. Pass-Stmt is a no-op placeholder.
- *Assertions.* Assert-Stmt records $\neg(\pi^{\Gamma} \wedge \pi_{\text{asrt}}) \vee \mathcal{E}$ in Π.
- *Assignments.* Assign (New) stores the evaluated expression for a new variable. For existing variables, runtime-dependent type changes are forbidden. If Θ marks the type mutable (Assign (Existing, Mut)) the symbolic value is overwritten; otherwise

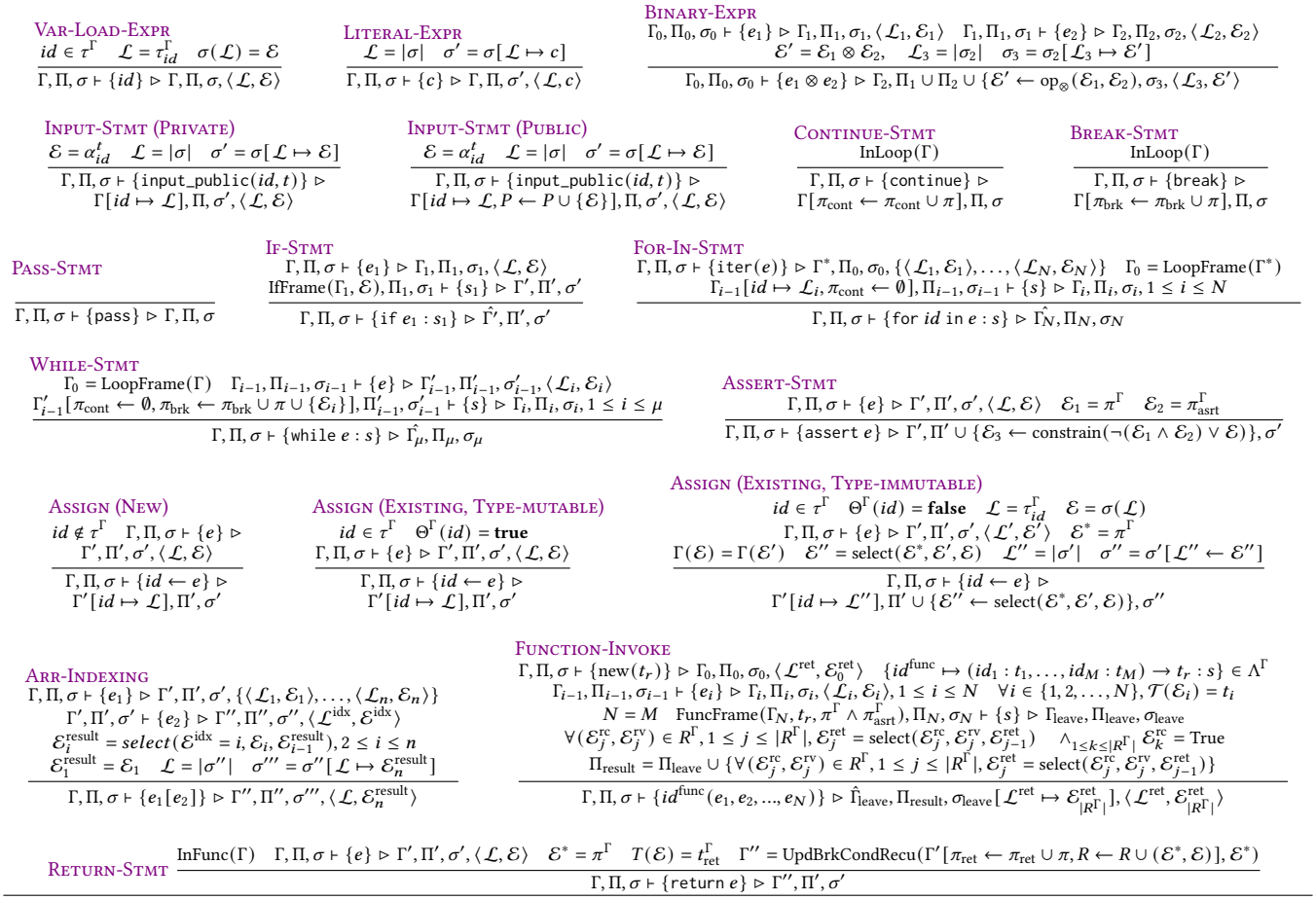

Figure 9: Examples of Operational Semantics for Zinnia's Symbolic Execution Compiler.

(Assign (Existing, Immut)) type equality is checked and `select` is used under the current path condition; see Section 4.6.

- *Array Manipulations.* Arr-Indexing and Arr-Updating operate on a flattened representation. Indexing builds nested `selects`, simplified to a direct reference when $\mathcal{V}(\mathcal{E}^{\text{idx}})$ is static. Updates replace matching candidates with the new value; with a static index only the targeted element changes.
- *Function Invoke.* Function-Invoke retrieves the definition from Λ, pushes a `FuncFrame` (return type and candidate values), sets assertion enablement to $\pi^{\Gamma} \wedge \pi^{\Gamma}_{\text{asrt}}$, checks argument types, and evaluates the body. A nested `select` over R yields the return value; exceeding recursion limit μ raises an error.
- *Return.* Return-Stmt evaluates e, checks $\mathcal{T}(\mathcal{E})$ against the declared type, adds $(\mathcal{E}, \pi^{\Gamma})$ to R, updates π_{brk}, sets π_{ret}, and skips the remainder of the function.

By applying these rules over the entire AST of a Zinnia program, the final symbolic state Σ is obtained, which contains the complete compilation result. The accumulated IR statements Π in Σ form a sequence of IR opcodes. Compared to the source code, the IR alleviates the data dependency and control-flow dependency issues, as it is a flat sequence of statements. The IR statements are branch-free, scalar, and in static single assignment form, which can be further optimized and synthesized into a ZKP circuit.

Remark. 1. *Traditional symbolic execution suffers exponential path explosion as the number of branches grows [6]. Zinnia sidesteps this by recording only the data dependencies of each branch condition and body, then merging them into a single symbolic state with conditional expressions instead of enumerating every possible path. The total compilation cost is $O(n) + O(n \cdot T_{\text{res}}(n))$ where n is the number of branches, which is scalable to real-world analytical workloads. See detailed discussion and evaluation in appendix.*

4.5 Resolution Function

In the symbolic execution engine, we need to resolve symbolic expressions to concrete values when possible. A *resolution oracle* $\mathcal{V}$ in our compiler decides whether a given *symbolic expression* (a multivariate polynomial $f : \mathbb{F}_p^v \to \mathbb{F}_p$) is *constant* (i.e. $f(\mathbf{x})$ has the same value for every input $\mathbf{x}$). This is the well-known Polynomial Identity Testing [44] problem. Our approach to implement the oracle is a heuristic-with-fallback engine. A fixed set of fast syntactic simplifications, degree propagation, and interval analyses

resolves most real-world expressions in time $T_{\text{res}} = \max_{1 \le i \le k} T_i(\mathcal{E})$; crucially, each test may say "non-constant" or "unsure" but never "constant" in error, which effectively preserves soundness and works efficiently in practice.

4.6 Type System

Recall that the symbolic execution engine relies on a type inference function $\mathcal{T}$ to determine the type of each expression. This section details the design of our type system, including its static typing feature, data types, the type inference function, and type bindings.

4.6.1 Static Typing and Data Types. Unlike Python, our DSL is statically typed to allow the generation of ZKP circuits ahead of runtime. Developers first declare the types of their input variables, which are then used to infer the types of subsequent expressions in the program without requiring explicit type annotations. Zinnia supports a collection of data types, including integers, floats, booleans, lists, tuples, and multidimensional arrays. Also, it is worth noting that, for composite types, we explicitly distinguish them based on their lengths or shapes. For example, two arrays with different lengths are considered distinct types, even if their element types are the same.

4.6.2 Type Inference. While the language adheres to static typing, we aim to simplify the development process by obviating the need for explicit type annotations. We accomplish this through local type inference, detailed in Fig. 10. The type inference function for an expression $\mathcal{E}$ is denoted as $\mathcal{T}(\mathcal{E})$, and the type inference judgment is expressed as $\rho \vdash \{e\} \rhd t$, where ρ represents the typing environment, e is the expression undergoing type inference, and t is the inferred type.

$$\textsc{T-Lit}\ \frac{}{\rho \vdash \{c\} \rhd \texttt{Int}} \qquad \textsc{T-Var}\ \frac{x : t \in \rho}{\rho \vdash \{x\} \rhd t} \qquad \textsc{T-BinOp}\ \frac{x_1 : t_1 \in \rho \quad x_2 : t_2 \in \rho}{\rho \vdash \{x_1 \otimes x_2\} \rhd \mathcal{T}_{\otimes}(t_1, t_2)}$$

$$\textsc{T-UOp}\ \frac{x : t \in \rho}{\rho \vdash \{\odot x\} \rhd \mathcal{T}_{\odot}(t)} \qquad \textsc{T-List}\ \frac{x_i : t_i \in \rho \text{ for } i = 1, \ldots, n}{\rho \vdash \{\texttt{list}(x_1, x_2, \ldots, x_n)\} \rhd \texttt{List}[t_1, t_2, \ldots, t_n]}$$

$$\textsc{T-Array}\ \frac{\begin{array}{c} x_i : t_i \in \rho \text{ for } 1 \le i \le n \\ t = t_1 = t_2 = \ldots = t_n \end{array}}{\rho \vdash \{\texttt{array}(x_1, x_2, \ldots, x_n)\} \rhd \texttt{Array}[t; n]} \qquad \textsc{T-List-Index-A}\ \frac{\begin{array}{c} x : \texttt{List}[t_1, t_2, \ldots, t_n] \in \rho \\ i : \texttt{Int} \in \rho \quad 0 \le \mathcal{V}(i) < n \end{array}}{\rho \vdash \{x[i]\} \rhd t_i}$$

$$\textsc{T-Arr-Index}\ \frac{\begin{array}{c} x : \texttt{Array}[t; n] \in \rho \\ i : \texttt{Int} \in \rho \end{array}}{\rho \vdash \{x[i]\} \rhd t} \qquad \textsc{T-List-Index-B}\ \frac{\begin{array}{c} x : \texttt{List}[t_1, t_2, \ldots, t_n] \in \rho \quad i : \texttt{Int} \in \rho \\ \mathcal{V}(i) = \textbf{unknown} \quad t = t_1 = t_2 = \ldots = t_n \end{array}}{\rho \vdash \{x[i]\} \rhd t}$$

$$\textsc{T-Arr-Slice}\ \frac{\begin{array}{c} x : \texttt{Array}[t; n] \in \rho \quad i : \texttt{Int} \in \rho \\ j : \texttt{Int} \in \rho \quad 0 \le \mathcal{V}(j), \mathcal{V}(i) < n \end{array}}{\rho \vdash \{x[i : j]\} \rhd \texttt{Array}[t; j - i]} \qquad \textsc{T-List-Slice}\ \frac{\begin{array}{c} x : \texttt{List}[t_1, t_2, \ldots, t_n] \in \rho \\ 0 \le \mathcal{V}(j), \mathcal{V}(i) < n \\ i : \texttt{Int} \in \rho \quad j : \texttt{Int} \in \rho \end{array}}{\rho \vdash \{x[i : j]\} \rhd \texttt{List}[t_{i+1}, \ldots, t_j]}$$

Figure 10: Examples of Type Inference Rules. For brevity, `Float` rules are analogous to `Int`, and `Tuple` rules to `List`.

While most of these rules are straightforward, we highlight a few key aspects. The type of an indexed expression in list indexing can be inferred in two ways, contingent on whether the index is statically known. In T-List-Index-A, the index's compile-time availability enables precise type inference. Conversely, in T-List-Index-B, the index is unknown until runtime, necessitating a conservative inference approach. In this scenario, all elements within the list must possess the same type; otherwise, the compiler cannot statically determine the resulting type. For the type inference of slicing operations, we mandate that both the start and end points be statically known, thereby allowing the determination of the resulting slice's shape.

4.6.3 Type Binding. In statically typed languages, a variable's type is fixed at compile time, while in interpreted languages it can change at runtime. Some statically typed languages (e.g., Rust) support variable shadowing, allowing a variable name to be re-bound to a different type for flexibility.

This flexibility, however, introduces challenges in maintaining type consistency. A variable's type may become path-dependent, leading to inconsistencies across branches. Fig. 11 illustrates this issue: starting from a singleton list `[0]`, a conditional branch appends an element, changing the list's length, and thus its type, since each list length has a distinct type. If the branch condition depends on input data, the list's type becomes data-dependent. The compiler then encounters conflicting types across execution paths and raises an error, undermining static guarantees.

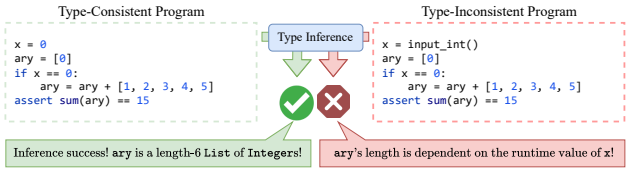

Figure 11: Type-Consistent and Inconsistent Examples.

To determine whether type re-binding is safe, we introduce the mutability function Θ. Alg. 1 computes $\Theta^{\Gamma}(id)$ for each assignment: (1) variables declared in the current (non-loop) scope are mutable where execution will not revisit old values; (2) in loops or when the variable is declared in a parent scope, mutability depends on the path condition. If the condition is not statically decidable, we conservatively treat the variable as immutable.

We now revisit the operational semantics for assignments as described in Sec. 4.4. In Assign (Existing, Mut), Θ determines that the variable is mutable, allowing the type to change. In contrast, Assign (Existing, Immut) returns immutable: the assignment is allowed only if the new value retains the original type (as resolved by `select`); otherwise, the compiler raises a type error.

5 Data Analytics Module

Real-world data analytics tasks often require support for real numbers, non-linear functions, and manipulation of multidimensional arrays, in addition to the programming framework introduced in Sec. 4.

5.1 Arithmetic Operations

Arithmetic circuits operate over a large prime field $\mathbb{F}_p$, in which only the operations of addition and multiplication are allowed.

Algorithm 1: The Function $\Theta^{\Gamma}(id)$

Input: identifier id, environment Γ
Output: {**true**, **false**}

```
if Γ is IfFrame then
    if id ∈ Γ.τ then
        return true
    if V(⋀_{E ∈ π^Γ_if} E) = true then
        return Θ^Γ̂(id)
    return false
if Γ is LoopFrame then
    if V(⋀_{E ∈ π^Γ_cont ∪ π^Γ_brk} ¬E) = true then
        if id ∈ Γ.τ then
            return true
        return Θ^Γ̂(id)
    return false
if id ∈ Γ.τ then
    return true
return Θ^Γ̂(id)
```

To encode higher-level data types and computations, we reduce each operation to a combination of these two primitives and, when necessary, auxiliary constraints.

Integer and Integer Arithmetic. Every integer x is interpreted as its residue class in $\mathbb{F}_p$. Field addition and multiplication directly implement integer addition and multiplication mod p. Subtraction and negation similarly exploit the field's additive inverses. To assert that an element x lies in the range $[0, 2^k)$, Zinnia appends a range proof circuit using lookup tables that enforces

$$x = \sum_{i=0}^{k-1} b_i \, 2^i, \text{ where } b_i \in \{0, 1\}, \tag{2}$$

using only linear constraints and Boolean-value checks.

Real Number and Real-valued Arithmetic. To support signed real values $x \in [-(p-1)/2, (p-1)/2]$ at precision 2^{-s}, following prior work [32, 49], we introduce a quantization mapping.

$$\hat{x} = \lfloor x \cdot 2^s \rceil \mod p \in \mathbb{F}_p, \tag{3}$$

where values $\hat{x} \geq \frac{p-1}{2}$ are interpreted as negative. Field addition and multiplication on the quantized $\hat{x}$ yield exact real-arithmetic up to rounding, and the original real is recovered by dequantization:

$$x \approx \begin{cases} \hat{x} \cdot 2^{-s} & x \leq \frac{p-1}{2} \\ (\hat{x} - p) \cdot 2^{-s} & x > \frac{p-1}{2} \end{cases} \tag{4}$$

Correctness is enforced by augmenting the circuit with a range-proof on $\hat{x}$ (using the same bit-decomposition technique as for integers) together with explicit bounds ensuring that the rounding error satisfies $|x - \hat{x}\, 2^{-s}| \leq 2^{-s-1}$.

5.2 Non-linear Functions

Functions such as e^x, $\ln(1+x)$, or $\sin x$ are replaced by a truncated Taylor series $f(x) \approx \sum_{i=0}^{d} a_i \, x^i$, where the degree d is chosen so that the Lagrange remainder R_{d+1} on the domain of interest is below a target ε. At proof time, the circuit evaluates $\sum_{i=0}^{d} a_i x^i$ using exactly d multiplications and d additions, thereby realizing the non-linear function within the prescribed error bound. We present an overview of supported non-linear functions in our framework in Table 2.

Table 2: Built-in Non-linear Functions.

Exponential and Logarithmic	`log`, `exp`, `log2`, `log10`, `sqrt`
Trigonometric	`sin`, `cos`, `tan`, `sec`, `csc`, `cot`
Inverse Trigonometric	`asin`, `acos`, `atan`
Hyperbolic	`sinh`, `cosh`, `tanh`

5.3 Multidimensional Arrays

Table 3: Examples of Multidimensional Array Operators.

Indexing	$arr[i_1, \ldots, i_n] \leftarrow val$, $id \leftarrow arr[i_1, \ldots, i_n]$
Slicing	$arr[i_1 : i_2 : i_3, \ldots] \leftarrow arr'$, $id \leftarrow arr[i_1 : i_2 : i_3, \ldots]$
Creation	`asarray`, `linspace`, `arange`, `eye`, `zeros`, `ones`, …
Aggregation	`sum`, `product`, `mean`, `argmax`, `max`, …
Computation	`mat_mul`, `dot`, `multiply`, …
Manipulation	`reshape`, `stack`, `split`, `transpose`, `repeat`, …

To handle multidimensional arrays of rank d, we flatten indices into a single field element via $\text{idx}(i_1, \ldots, i_d) = \sum_{j=1}^{d} \left(i_j \cdot \prod_{k<j} n_k \right)$, where n_j is the size of dimension j. Access to the ℓth element then becomes a constrained selection gate that enforces consistency between the flattened index and the chosen array entry. Higher-level operations like matrix multiplication or axis-wise aggregations are decomposed into scalar multiplications and additions over these flattened representations. We report the supported array operators in Table 3.

6 Circuit Synthesis

Zinnia specifies a suite of IR to express both data flow and arithmetic semantics in the compiler. Every IR instruction performs a scalar operation (either on finite-field integers or on real numbers) and produces a single scalar result. The instruction set covers basic arithmetic (+, −, ×, /, %, //), bitwise logic (`AND`, `OR`, `XOR`), comparisons (=, ≠, <, >, ≤, ≥), casts between integer and real domains. For advanced non-linear functions, we implement them as intrinsic instructions that invoke the underlying arithmetic circuit library for efficient implementation.

In Zinnia, we implement a circuit synthesizer that translates the IR into arithmetic circuits with two backends: Halo2-lib [4] and Noir [40]. Halo2-lib is a library for building PLONK arithmetic circuits. Halo2-lib exposes ready-made gadgets for finite-field arithmetic, comparisons, and nonlinear functions, so our synthesis layer simply invokes these primitives rather than manually wiring gates. Noir is a high-level pDSL that works with ULTRAHONK, a more flexible and efficient arithmetic circuit protocol. The synthesizer emits Noir code that directly corresponds to the IR opcodes and offloads the circuit generation to Noir's backend. We anticipate that this modular design will facilitate future integration with other proving backends and allow us to adapt to evolving ZKP systems.

7 Evaluation

We aim to answer the following questions in our evaluation:

- **RQ1: Usability.** Can Zinnia effectively reduce development effort in real-world data analytics tasks?
- **RQ2: Performance.** Does Zinnia manifest performance advantages over zkVM systems and SNARK-based DSLs?
- **RQ3: Ablation Study.** How different techniques in Zinnia contribute to its overall performance?

To answer **RQ1**, we manually implement 25 programming tasks from various domains using Zinnia as well as the four industrial-strength ZKP systems (Halo2, Noir, RISC0, SP1, and Cairo), and compare the complexity of the code. Then, we conduct a user study to understand the workloads for developers to implement three programming tasks using Zinnia and Cairo, a Rust-like zkVM system with established usability. To answer **RQ2**, we compare Zinnia against same baseline ZKP systems to evaluate the proof generation and verification performance as well as the circuit size. Finally, to answer **RQ3**, we conduct an ablation study to understand how our symbolic execution engine and various compilation optimizations contribute to the overall performance.

In addition, we also provide evaluation results for compilation scalability, comprehensive case studies, experimental setups, and detailed evaluation results (including proof size) in the appendix. Below, we first introduce the experimental setup and then present the results for each research question in turn.

7.1 Experimental Setup

Datasets. We prepare 25 programming tasks from four sources: **ML**, **LeetCode**, **DS-1000** and **Crypt** to mirror the data analytics applications, from machine learning to data processing, statistical analysis and general-purpose programming. **ML** consists of three machine learning tasks: Neuron, K-Means, and Linear Regression. **LeetCode (LC)** consists of ten programming tasks randomly selected from LeetCode [33] and covers five categories (Array, Dynamic Programming, Graph, Math, Matrix), with two difficulty levels (one easy, one medium) per category. **DS-1000 (DS)** consists of ten data science tasks randomly sampled from DS-1000 [29], a corpus of 1000 data science tasks in Python. **Crypt (CRY)** consists of two cryptographic tasks: Baby Jubjub ECC [8] and Poseidon Hash [22]. One of the authors implement these in Zinnia and baseline ZKP systems, and another independently reviews them to ensure correctness.

User Study. We recruited six participants who are proficient in Python and Rust but with no prior ZKP experience and split them evenly into a Zinnia group and a Cairo group. Cairo serves as a baseline due to its established usability in the zkVM landscape that can directly compile Rust-like code into ZKP circuits. We exclude low-level frameworks (e.g., cDSLs and pDSLs) as they require substantial expertise to our participants. Each participant completed a brief tutorial on their assigned system and passed a comprehension quiz before proceeding. They then had 60 minutes to implement three elementary-level programming tasks:

- ① **Primality Testing.** Given a number, determine if it is prime.
- ② **Staircase Climbing.** Given a number of steps, count the number of distinct ways to climb the staircase with 1 or 2 steps at a time.
- ③ **Path Existence.** Given a directed graph represented as an adjacency matrix, determine if there exists a path from a source node to a target node.

Table 4: Benchmark Implementation Report.

Dataset		ML	LC	DS	CRY
No. Program Constructs (Maximum)	Loops	5	5	4	0
	Break / Continue	0	1	1	0
	Branches	1	3	1	0
Average LoC	Zinnia	24.0	13.0	5.90	15.0
	Halo2	67.0	42.5	31.5	62.5
	Noir	N/A	24.6	18.5	19.0
	RISC0/SP1	72.0	48.0	36.7	57.0
	Cairo	N/A	29.1	19.7	18.0

Table 5: Task Completion Time and Correctness.

Participant		Time (in minutes) T1	T2	T3	Correctness T1	T2	T3	Tot. Time
Zinnia Group	P1	15	8	18	✓	✓	✓	41
	P2	12	6	25	✓	✓	✓	43
	P3	5	7	10	✓	✓	✓	22
Zinnia Avg.		10.67	7.00	17.67	100%	100%	100%	
Cairo Group	P4	5	13	45	✓	✓	✓	63
	P5	16	13	**60**	✓	✓	×	89
	P6	11	15	**60**	✓	✓	×	86
Cairo Avg.		10.67	10.33	55.00	100%	100%	33.3%	

We record task completion time and assess the correctness of completed tasks.

Environment. All experiments are conducted on a server with dual Intel(R) Xeon(R) Gold 6444Y CPUs @ 3.60GHz and 256GB RAM. Multithreading (64 threads) is enabled for zkVMs and all baselines run with their default configurations.

7.2 RQ1: Usability

In Sec. 4.1, we present several cases where Zinnia effectively implements data analytics tasks that are difficult to express in existing ZKP systems. Here, we provide a more thorough evaluation of Zinnia through the code length and complexity of implementing real-world data analytics tasks, as well as a user study.

We implement the 25 programming tasks described in Sec. 7.1 and compare the code complexity against baselines. We exclude exclude ML and some DS tasks for Noir and Cairo, as they lack real-valued arithmetic support. We report the maximal control flows and the average LoC (lines of code) in Table 4. These tasks involve complex control flows with loops and conditionals, with the most complex case consisting of 5 nested loops, and multiple conditional statements. Therefore, encoding these tasks are inherently challenging in low-level ZKP systems such as Halo2, while Zinnia significantly simplifies the process. In particular, Zinnia delivers 2–3× shorter code than the baseline systems on average, with a peak reduction of over 6× for DS-1000 tasks. The rationale is multi-fold: first, Zinnia supports data-dependent dynamic programming constructs that allow developers to express complex logic directly, and avoid the low-level constraint plumbing required by Halo2 or the static program structure of Noir; second, it offers a rich suite of native arithmetic and array operations to streamline data-manipulation pipelines, whereas developers using zkVMs have to manually implement these operations from scratch.

We further launch a user study to understand the usability of Zinnia for developers in the wild. We compare the completion time and correctness of the tasks between Zinnia and Cairo. We report the program correctness and completion time for each participant in Table 5, and the correctness is determined by passing all test cases. Compared to the baseline, participants using Zinnia exhibit a trend of faster task completion (p-value ≤ 0.05 with Mann-Whitney U Test), especially for Tasks 2 and 3. In terms of program correctness, most tasks were successfully implemented by our participants. However, Participants 5 and 6 from the baseline group were unable to complete Task 3 within the allocated time. In the post-study survey, the participants expressed that they found Zinnia's dynamic programming constructs intuitive and easy to use, while they struggled with Cairo's static control flow and complex array operations.

Answer to RQ1: Zinnia is capable of implementing a wide range of data analytical applications with significantly shorter code than baseline systems. The user study further validates its usability with reduced task completion times and improved correctness compared to Cairo.

7.3 RQ2: Performance

In this section, we evaluate the performance of Zinnia and compare it against five baseline ZKP systems: Cairo, RISC0, SP1, Halo2, and Noir. We focus on the proving and verifying time, as well as the circuit size (measured by constraint count). For Cairo, RISC0 and SP1, we only report the proving and verifying time, as they employ a different arithmetic circuit model that is not directly comparable to Zinnia. For Halo2 and Noir, we report the constraint count, proving and verifying time under the same arithmetic circuit model as Zinnia. Notably, tasks involving real-valued arithmetic are excluded for Noir and Cairo as they do not support that.

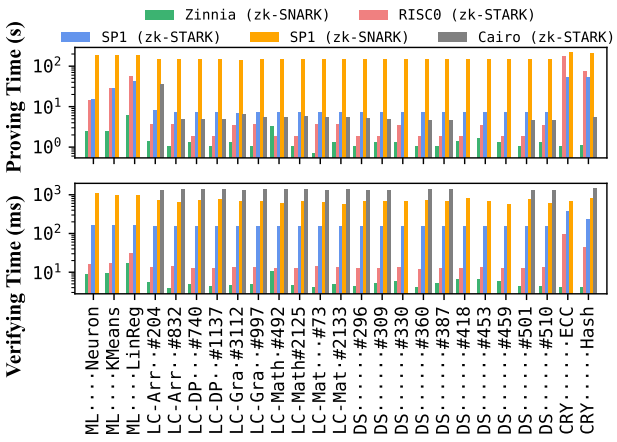

Figure 12: Comparison of Proving and Verifying Time against Cairo, RISC0 and SP1. *On average, Zinnia outperforms RISC0, SP1, SP1's SNARK prover and Cairo by 25.4×, 20.3×, 245.4× and 5.8× in proving and by 5.0×, 45.1×, 191.7×, and 290.4× in verifying, respectively.*

Fig. 12 compares the proving and verifying time of Zinnia, Cairo (STARK), RISC0 (STARK), SP1 (STARK), and SP1's SNARK prover. The results show that zkVM solutions (Cairo, RISC0, SP1, and SP1's SNARK prover) exhibit significantly longer proving and verifying time compared to Zinnia (over 25.4× and 5.0×, respectively). This lag is expected. First, as zkVM-based systems, both Cairo, RISC0 and SP1 relies on costly per-instruction CPU emulation to execute programs, where each instruction emulation maintains a large number of registers, memory, and program counters. This overhead is amplified when wrapping these proofs in the SNARK prover, which further increases the proving time by 5.8× to 245.4× on average. In contrast, Zinnia alleviates this overhead by directly generating arithmetic circuits to represent the program logic, which is more efficient for ZKP systems.

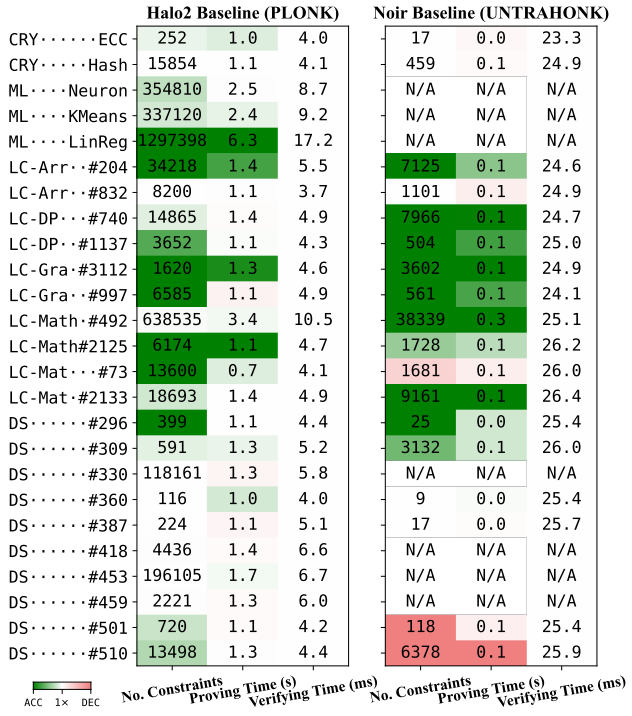

Figure 13: Circuit Performance Comparison. *"ACC"/"DEC" means accelerate and decelerate, respectively. The proving time is measured in* ***seconds****, while the verification time is measured in* ***milliseconds****. N/A denotes tasks excluded for Noir due to the lack of real-valued support.*

We report the performance of Zinnia against Halo2 (as a representative cDSL) and Noir (as a representative pDSL) in Fig. 13. On many tasks, Zinnia delivers substantial performance improvements over Halo2 and Noir, with an average proving time of 4.9% and 14.5% faster than Halo2 and Noir, respectively and an average saving of 19.3% and 22.8% in constraint count, respectively. We refrain from comparing the verification time, as it is very fast (typically under a few milliseconds) and dominated by system-level factors rather than the circuit complexity.

Suboptimal Cases in Noir. While Zinnia generally outperforms other systems, it does not achieve the best performance on all tasks. In particular, Noir performs better on three array-indexing tasks (LC #73, DS #501, and DS #510). This advantage arises from Noir's ULTRAHONK-specific compiler, which, as a industrial-strength tool, can leverage native array length and bounds semantics to produce tighter constraints. In contrast, Zinnia lowers each indexing operation into a field-wise multiplexer over all possible positions, which introduces additional overhead. This reflects a deliberate design trade-off in Zinnia: we prioritize backend-agnostic generality in the IR and ensure portability across proof systems that may not support native array semantics. While this design sacrifices certain backend-specific optimizations, it improves overall compatibility. In future work, we plan to incorporate more backend-specific optimizations by extending the IR with tailored intrinsics where appropriate.

Answer to RQ2: On a diverse set of ML, programming, data science and cryptographic tasks, Zinnia delivers much faster proof-generation and verification than zkVMs like Cairo, RISC0, and SP1 without sacrificing programming flexibility. It produces compact arithmetic circuits that match or exceed Halo2 and Noir implementations, demonstrating Zinnia's effectiveness in generating optimized circuits.

7.4 RQ3: Ablation Study

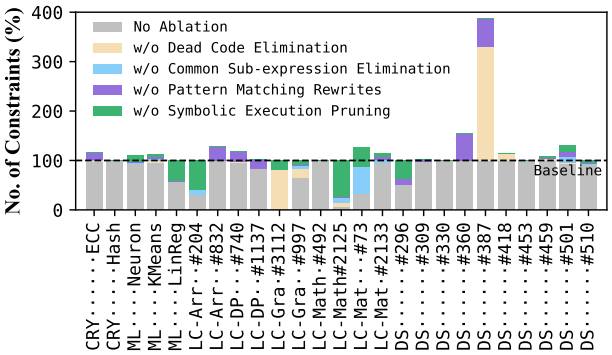

Figure 14: Ablation Study on Circuit Performance.

To understand the mechanisms underlying the performance achieved by Zinnia, we conduct an ablation study. In summary, there are two key components that contribute to the performance of Zinnia: the symbolic execution engine and the compiler optimizers. The symbolic execution engine performs path pruning and data dependency analysis to reduce the number of constraints in the circuit, while the optimizers apply various techniques to eliminate redundant constraints and simplify circuit patterns. To evaluate their impact, we disable them individually and measure the performance through the constraint count. We present the results in Fig. 14.

As anticipated, disabling these optimizations yields a considerable surge in constraints. Notably, for the majority of tasks, the constraints count exceed the hand-crafted Halo2 baseline presented in Sec. 7.3 by 21.3% on average. It indicates that simply translating the IR (or program) into arithmetic circuits without finer-grained program analysis and optimization could lead to inefficient circuits and cannot match the performance of hand-crafted circuits. In contrast, with these techniques, Zinnia achieves substantial reductions in constraint count.

Insights. We now inspect the factors driving these gains. On `LC-#204` (prime counting), we cut circuit size by 68.6% by using our symbolic execution engine to analyze the program and to detect that "number of primes below N" (which is an intermediate result in the full program) is a constant; thus, it optimizes the sieve of Eratosthenes algorithm to precompute the result off-circuit and eliminate the expensive computation. Similarly, for `LC-#3112`, Zinnia's compilation optimization technique identifies and eliminates the unused intermediate distance values in the Floyd-Warshall all-pairs shortest-path algorithm. Likewise, in `LC-#2125`'s brute-force laser counting routine, we detect and remove constant computations and always satisfiable constraints, which alleviates unnecessary overhead in the circuit. In `DS-1000#387`, Zinnia's compilation optimization technique detects and removes redundant copying gates in the circuit that are incurred by multidimensional reshaping. Across these tasks, Zinnia's symbolic execution engine as well as its compilation optimization techniques consistently identify and address optimization opportunities, which contribute to substantial reductions. We also review cases where Zinnia produces less efficient circuits. In particular, tasks like `Neuron`, `K-Means`, `LC-#492` and several `DS` examples perform simple arithmetic and simple data manipulation without complex control flow. In these cases, the symbolic execution engine and compilation optimizers do not yield significant gains, as the circuits are already close to optimal.

Answer to RQ3: On a diverse set of benchmarks, Zinnia effectively reduce circuit complexity by a combination of IR-level optimizations and symbolic execution's path pruning, achieving an average reduction of 21.3% compared to the ablation baseline.

8 Related Work

We discuss ZKP data analytics and programming interfaces in Sec. 1 and Sec. 2.2. In this section, we focus on other privacy-enhancing technologies (PETs) and general programming interfaces for data analytics.

Other PETs for Data Analytics. In addition to ZKPs, a variety of PETs have been developed to protect sensitive data during analytics. These include differential privacy (DP), secure multi-party computation (MPC), homomorphic encryption (HE), federated learning (FL), trusted execution environments (TEE), and oblivious databases. DP [16] provides a rigorous framework for adding calibrated noise to query results, and has been implemented in systems such as PINQ [36], wPINQ [42, 43], and DJoin [38], as well as extended to support rich SQL queries [15, 27]. Cryptographic approaches like MPC [51] and HE [20] effectively enable joint computation over encrypted data with systems like CryptDB [41] and MASQUE [34], albeit with performance costs. FL [35] empowers decentralized model training by keeping raw data on-device and exchanging only encrypted or

privacy-enhanced updates [19, 39]. TEE [5] leverages hardware enclaves to securely process sensitive data without exposing it to the host. Oblivious databases [18] utilize Oblivious RAM [21] to further conceal data access patterns. ZKPs complement these approaches by providing a mechanism to verify computations without revealing the underlying data, enabling a new class of verifiable analytics that can be performed on encrypted or privacy-preserving data.

Programming Interface for Data Analytics. Designing user-friendly programming interfaces for data systems has long been a central focus of the data management community. These interfaces aim to capture user intent and translate it into efficient execution plans. Systems such as LINQ [37], DryadLINQ [25], and epiC [26] have demonstrated the power of using high-level programming languages to express data transformations and automatically compile them into efficient execution pipelines. More recently, there has been growing interest in applying program synthesis techniques to lift imperative user code into specialized data processing APIs [2, 3]. Zinnia continues this line of work by leveraging a symbolic execution engine to formally reason about programs and generate optimized ZKP circuits.

9 Conclusion and Future Work

We introduced Zinnia, a general-purpose programming framework designed to address the challenges of developing ZKPs for data analytics. Zinnia's data-dependent language, symbolic execution compiler, and comprehensive data-analytic support in the framework enable high utility, expressiveness, and performance improvements over existing solutions.

Several promising directions remain for future work. First, Zinnia can be extended with features such as user-defined data types, string manipulation, and generics to further enhance its expressiveness and applicability. Additionally, we plan to broaden Zinnia's analytical capabilities by supporting more advanced modules, including graph algorithms and machine learning workloads.

References

[1] Harold Abelson and Gerald Jay Sussman. 1996. *Structure and interpretation of computer programs*. The MIT Press.

[2] Maaz Bin Safeer Ahmad and Alvin Cheung. 2017. Optimizing data-intensive applications automatically by leveraging parallel data processing frameworks. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1675–1678.

[3] Maaz Bin Safeer Ahmad and Alvin Cheung. 2018. Automatically leveraging mapreduce frameworks for data-intensive applications. In *Proceedings of the 2018 International Conference on Management of Data*. 1205–1220.

[4] axiom crypto. 2025. Halo2-lib. https://github.com/axiom-crypto/halo2-lib.

[5] Sumeet Bajaj and Radu Sion. 2014. TrustedDB: A Trusted Hardware-Based Database with Privacy and Data Confidentiality. *IEEE Transactions on Knowledge and Data Engineering* 26, 3 (2014), 752–765. doi:10.1109/TKDE.2013.38

[6] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3, Article 50 (May 2018), 39 pages. doi:10.1145/3182657

[7] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. 2022. Circom: A circuit description language for building zero-knowledge applications. *IEEE Transactions on Dependable and Secure Computing* 20, 6 (2022), 4733–4751.

[8] Marta Bellés-Muñoz, Barry Whitehat, Jordi Baylina, Vanesa Daza, and Jose Luis Muñoz-Tapia. 2021. Twisted edwards elliptic curves for zero-knowledge circuits. *Mathematics* 9, 23 (2021), 3022.

[9] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2014. Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture. In *23rd USENIX Security Symposium (USENIX Security 14)*. USENIX Association, San Diego, CA, 781–796. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/ben-sasson

[10] Manuel Blum, Paul Feldman, and Silvio Micali. 1988. Non-interactive zero-knowledge and its applications. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*. 103–112.

[11] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019).

[12] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 560–574.

[13] Collin Chin, Howard Wu, Raymond Chu, Alessandro Coglio, Eric McCarthy, and Eric Smith. 2021. Leo: A programming language for formally verified, zero-knowledge applications. *Cryptology ePrint Archive* (2021).

[14] Electric Coin Co. 2025. halo2. https://github.com/zcash/halo2.

[15] Wei Dong, Zijun Chen, Qiyao Luo, Elaine Shi, and Ke Yi. 2024. Continual observation of joins under differential privacy. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[16] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. 2006. Our data, ourselves: Privacy via distributed noise generation. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 486–503.

[17] Jacob Eberhardt and Stefan Tai. 2018. ZoKrates - Scalable Privacy-Preserving Off-Chain Computations. In *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. 1084–1091. doi:10.1109/Cybermatics_2018.2018.00199

[18] Saba Eskandarian and Matei Zaharia. 2019. ObliDB: oblivious query processing for secure databases. *Proc. VLDB Endow.* 13, 2 (Oct. 2019), 169–183. doi:10.14778/3364324.3364331

[19] Jie Fu, Yuan Hong, Xinpeng Ling, Leixia Wang, Xun Ran, Zhiyu Sun, Wendy Hui Wang, Zhili Chen, and Yang Cao. 2024. Differentially private federated learning: A systematic review. *arXiv preprint arXiv:2405.08299* (2024).

[20] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.

[21] Oded Goldreich and Rafail Ostrovsky. 1996. Software protection and simulation on oblivious RAMs. *Journal of the ACM (JACM)* 43, 3 (1996), 431–473.

[22] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. 2021. Poseidon: A new hash function for {Zero-Knowledge} proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 519–535.

[23] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*. Springer, 305–326.

[24] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.

[25] Michael Isard and Yuan Yu. 2009. Distributed data-parallel computing using a high-level programming language. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 987–994.

[26] Dawei Jiang, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, and Sai Wu. 2014. epiC: an extensible and scalable system for processing big data. *Proceedings of the VLDB Endowment* 7, 7 (2014), 541–552.

[27] Noah Johnson, Joseph P Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.

[28] Succinct Labs. 2025. SP1. https://github.com/succinctlabs/sp1.

[29] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: a natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 756, 27 pages.

[30] Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, and Bhaskar Krishnamachari. 2024. A Survey on the Applications of Zero-Knowledge Proofs. *arXiv preprint arXiv:2408.00243* (2024).

[31] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. Zksql: Verifiable and efficient query evaluation with zero-knowledge proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023).

[32] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 2968–2985. doi:10.1145/3460120.3485379

[33] LeetCode LLC. 2025. LeetCode. https://leetcode.com/.

[34] Qiyao Luo, Yilei Wang, Ke Yi, Sheng Wang, and Feifei Li. 2023. Secure sampling for approximate multi-party query processing. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–27.

[35] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.

[36] Frank D McSherry. 2009. Privacy integrated queries: an extensible platform for privacy-preserving data analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 19–30.
[37] Erik Meijer, Brian Beckman, and Gavin Bierman. 2006. Linq: reconciling object, relations and xml in the. net framework. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*. 706–706.
[38] Arjun Narayan and Andreas Haeberlen. 2012. {DJoin}: Differentially private join queries over distributed databases. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. 149–162.
[39] Maxence Noble, Aurélien Bellet, and Aymeric Dieuleveut. 2022. Differentially private federated learning on heterogeneous data. In *International conference on artificial intelligence and statistics*. PMLR, 10110–10145.
[40] Noir. 2025. Noir. https://noir-lang.org/.
[41] Raluca Ada Popa, Catherine MS Redfield, Nickolai Zeldovich, and Hari Balakrishnan. 2011. CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the twenty-third ACM symposium on operating systems principles*. 85–100.
[42] Davide Proserpio, Sharon Goldberg, and Frank McSherry. [n. d.]. wPINQ: Differentially-Private Analysis of Weighted Datasets. ([n. d.]).
[43] Davide Proserpio, Sharon Goldberg, and Frank McSherry. 2012. Calibrating data to sensitivity in private data analysis. *arXiv preprint arXiv:1203.3453* (2012).
[44] Nitin Saxena. 2009. Progress on Polynomial Identity Testing. *Bull. EATCS* 99 (2009), 49–79.
[45] scipr lab. 2025. libsnark. https://github.com/scipr-lab/libsnark.
[46] Robert W Sebesta, Soumen Mukherjee, and Arup Kumar Bhattacharjee. 1999. *Concepts of programming languages*. Vol. 7. Addison-Wesley Reading, Massachusetts.
[47] Tobin South, Alexander Camuto, Shrey Jain, Shayla Nguyen, Robert Mahari, Christian Paquin, Jason Morton, and Alex'Sandy' Pentland. 2024. Verifiable evaluations of machine learning models using zkSNARKs. *arXiv preprint arXiv:2402.02675* (2024).
[48] StarkWare. 2025. Cairo. https://github.com/starkware-libs/cairo.
[49] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 501–518.
[50] Hao Wu, Changzheng Wei, Yanhao Wang, Li Lin, Yilong Leng, Shiyu He, Minghao Zhao, Hanghang Wu, Ying Yan, and Aoying Zhou. 2025. Zero-Knowledge Verifiable Graph Query Evaluation via Expansion-Centric Operator Decomposition. *arXiv preprint arXiv:2507.00427* (2025).
[51] Andrew C Yao. 1982. Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*. IEEE, 160–164.
[52] RISC Zero. 2025. RISC Zero. https://github.com/risc0/risc0.

A Scalability of Compilation

Symbolic execution often incurs considerable overhead, primarily due to the "path explosion" problem, where complexity scales exponentially (2^b for b conditional branches). This exponential growth stems from each conditional branch doubling the possible execution traces and forking the symbolic state space. Zinnia sidesteps this by recording only the data dependencies of each branch condition and body, then merging them into a single symbolic state with conditional expressions instead of enumerating every possible path. In this section, we provide both a theoretical analysis of the compile-time complexity of our approach and an empirical evaluation of its performance on our benchmarks.

A.1 Complexity Analysis

We first define the metrics we use to analyze the complexity:

Def. 1 (Program Metrics). *Let P be a program. We write*

- *$b(P)$ for the number of conditional branches in program P's AST. For simplicity of analysis, all loops are assumed to be pre-unrolled into nested conditionals as they're bounded by either the length on the iterable object or by a predefined constant μ.*
- *$n(P)$ for the length of the IR produced by the compiler, measured by the number of statements.*

Def. 2 (Translation Function). *Let $\mathcal{F} : \text{AST} \to \text{IR}^*$ be the structural recursion that, for every node α in the AST, emits a bounded sequence $\mathcal{F}(\alpha)$ of IR opcodes on the semantics of α.*

Lem. 1 (Local Boundedness). *There exists a universal constant $c > 0$ such that for every AST node α, $|\mathcal{F}(\alpha)| \le c$.*

Proof. The compiler contains a finite set of operational semantics for the translation. The maximum length serves as the constant c. □

Lem. 2 (Linear IR Size). *For every program P, $n(P) = \sum_{\alpha \in \text{AST}(P)} |\mathcal{F}(\alpha)| \le c \cdot |\text{AST}(P)|$.*

Proof. Immediate from Lemma 1. □

Thm. 1 (Translation Complexity). *Let $b(P)$ be as in Definition 1. The time required to translate P (excluding resolution calls) is $O\big(|\text{AST}(P)|\big) = O\big(b(P)\big)$.*

Proof. A depth-first traversal visits each AST node exactly once and emits $\mathcal{F}(\alpha)$. By Lemma 1, emission costs $O(1)$ per node; hence the total cost is proportional to the number of nodes. Since every branch contributes at least one node, $|\text{AST}(P)| \in \Theta\big(b(P)\big)$. □

Cor. 1 (Overall Compile-Time Bound). *For a program P with resolution approach $\mathcal{V}_{\text{res}}$, the compile-time complexity of our symbolic-execution-based compiler is*

$$T(P) = O\big(b(P)\big) + O\big(n(P) \cdot T_{\text{res}}\big(n(P)\big)\big)$$

Proof. Combine Theorem 1 with previously defined T_{res}. □

We conclude that the compile-time complexity of our symbolic execution approach is bounded by a linear function of the number of conditionals, $b(P)$, and the heuristic resolution time $T_{\text{res}}\big(n(P)\big)$ for the IR size $n(P)$. Given that $n(P)$ also scales linearly with $b(P)$, the overall complexity simplifies to $O\big(b(P)\big) + O\big(b(P) \cdot T_{\text{res}}\big(b(P)\big)\big)$, where $b(P)$ is the number of branches in the program. This polynomial complexity is far more manageable than the exponential complexity of traditional symbolic execution: as previously clarified, by reusing and combining symbolic contexts rather than duplicating them, we convert an exponential exploration problem into a polynomial-time process. Specifically, if all heuristics exhibit linear performance, the total complexity becomes $O(b^2(P))$. This quadratic complexity is consistent with that of typical compilers, making our approach well-suited for practical applications, even for complex programs with numerous branches and input variables.

A.2 Empirical Evaluation

To further validate our scalability, we conducted an empirical evaluation of Zinnia's compile-time performance across various benchmarks. We report the compilation time for each benchmark in Table 6.

Table 6: Measured Zinnia Compilation Time.

Benchmark	Time (s)	Benchmark	Time (s)
ML-Neuron	6.0	DS-#296	0.0
ML-KMeans	4.2	DS-#309	0.0
ML-LinReg	13.0	DS-#330	0.0
LC-#204	25.3	DS-#360	0.0
LC-#832	0.8	DS-#387	0.0
LC-#740	0.4	DS-#418	0.0
LC-#1137	0.1	DS-#453	0.0
LC-#3112	6.1	DS-#459	0.0
LC-#997	0.4	DS-#501	0.0
LC-#492	13.7	DS-#510	0.3
LC-#2125	2.6	CRY-ECC	0.0
LC-#73	2.4	CRY-Poseidon	0.0
LC-#2133	0.2		

Overall, we observe that Zinnia's compilation times are generally modest, with most benchmarks completing in just a few seconds. Data science (DS) tasks, particularly those with simpler control flows, compile almost instantly. In contrast, machine learning (ML) benchmarks take longer due to their computational complexity, while LeetCode (LC) tasks show more variation depending on the specific algorithm.

We further examined the benchmarks with longer compilation times. For example, `ML-LinReg` performs linear regression using a large loop to update model parameters via gradient descent, which increases compilation time. `LC-#204` implements an $O!\big(n^2\big)$ prime sieve for $n < 1000$, totaling 1,000,000 iterations, leading to similarly higher compile times. `LC-#492` also involves extensive looping (1,000 iterations) to test number divisibility.

Despite these cases involving large iterative computations, their compile times remain well within manageable bounds, aligning with our theoretical scalability guarantees for Zinnia.

B Case Study

In addition to the cases in the *Programming Model* section, we present more case studies complementing the main paper to demonstrate the expressiveness of Zinnia. While this showcase is not exhaustive, it highlights key examples of its capabilities. Readers will notice that these cases closely resemble programming in Python, which is a deliberate design choice aimed at making Zinnia intuitive for data analysts, without the need for extensive training in ZKPs. We provide a brief overview of our case studies in Table 7.

Table 7: Case Studies for each Zinnia's Feature.

Constructs	Dynamic Loop	B.1
	Recursion	B.2
	Func Return	B.5
	Index & Slice	B.3, B.7
Data Ops.	Real number	B.1, B.6, B.7
	Non-linear Op	B.6
	Multidim-Arr	B.1, B.3, B.7
	Multidim-Op	B.1, B.7

B.1 Linear Regression

```
@zk_circuit
def linear_regression(
    training_x: NDArray[float, 10, 2],
    training_y: NDArray[float, 10],
    testing_x: NDArray[float, 2, 2],
    testing_y: NDArray[float, 2]
):
    weights = np.zeros((training_x.shape[1], ))
    bias, m = 0, float(len(training_y))
    for _ in range(100):
        predictions = np.dot(training_x, weights) + bias
        errors = predictions - training_y
        dw = (1.0 / m) * np.dot(training_x.T, errors)
        db = (1.0 / m) * np.sum(errors)
        weights -= 0.02 * dw
        bias -= 0.02 * db
    test_predictions = np.dot(testing_x, weights) + bias
    test_diff = test_predictions - testing_y
    test_error = np.sum(test_diff * test_diff) / len(testing_y)
    assert test_error <= 0.1
```

Figure 15: Zinnia Code for Linear Regression Circuit.

This case study highlights Zinnia's proficiency in numerical computations and array operations, particularly in the context of machine learning tasks. We demonstrate a linear regression model implemented using gradient descent in Fig. 15. The code initializes weights and bias, then iteratively updates them based on the gradient descent formula. The error computation is also included. With the built-in real number support and multidimensional array operations such as mat-mul, Zinnia simplifies the implementation of this machine learning task in ZKPs, making it more accessible to data analytical users with limited zero-knowledge programming experience.

```
@zk_circuit
def is_prime(number: int, prime: bool):
    assert 0 <= number <= 10000
    if number < 2:
        assert not prime
    else:
        result = True
        for i in range(2, 101):
            if i * i > number:
                break
            if number % i == 0:
                result = False
        assert prime == result
```

Figure 16: Zinnia Code for Primality Testing.

B.2 Test Prime Number

In this case study, we implement a ZKP circuit using Zinnia to perform integer primality checking. Fig. 16 shows an alternative implementation that relies on loop controls such as `break` and `continue`, rather than solely on functions as presented in the main paper. Given that the input is bounded by 10,000, the code first asserts that numbers less than 2 are not prime. It then iterates from 2 to 100, checking for divisibility. To ensure computational efficiency, a `break` statement exits the loop once the square of the iterator exceeds the input value.

This example highlights Zinnia's ability to handle data-dependent control flow: the loop can terminate early based on the input, rather than being statically unrolled or forced to iterate a fixed number of times. This flexibility enhances expressiveness compared to prior DSLs, which often impose rigid iteration structures.

B.3 Identify Zero Rows and Columns

```
@zk_circuit
def remove_0_rows_cols(input: NDArray[int, 5, 6],
                       result: NDArray[int, 3, 4]):
    zero_rows = []
    zero_cols = []
    for i in range(5):
        zero_rows += [all(input[i, :] == 0)]
    for i in range(6):
        zero_cols += [all(input[:, i] == 0)]
    # [The following code omitted for brevity...]
```

Figure 17: Zinnia Code for Zero Checking.

The task is to identify all zero rows and columns from a 2-dim array. Fig. 17 shows the implementation, where we append non-zero information to an empty list using a loop. Slicing and indexing are used to access rows and columns on a multidimensional array, a feature rare in prior DSLs.

This case also demonstrates Zinnia's variable shadowing feature. Recall that lists of different length are treated as different types, with our variable shadowing feature, users essentially "redefine" the original variable `zero_rows` each iteration with new type: a list whose length is 1 more. This allows users to change the type of variables defined in the outer or same scope, making code implementation more flexible.

B.4 Point Addition on Baby Jubjub ECC

```
@zk_chip
def baby_add(
    x1: int, y1: int, x2: int, y2: int) -> Tuple[int, int]:
    a = 168700
    d = 168696
    beta = x1 * y2
    gamma = y1 * x2
    delta = (-a * x1 + y1) * (x2 + y2)
    tau = beta * gamma
    x = (beta + gamma) * math.inv(1 + d * tau)
    y = (delta + a * beta - gamma) * math.inv(1 - d * tau)
    return x, y
```

Figure 18: ZINNIA Code for ECC Point Addition.

While ZINNIA abstracts away the complexities of finite field arithmetic, it still allows users to implement them for cryptographic needs. This case shows elliptic curve operations over finite field, specifically point addition on Baby Jubjub elliptic curve. The code in Fig. 18 shows a UDF baby_add that takes two points and returns a new point. To facilitate finite field arithmetic, ZINNIA provides `math.inv` to compute the inverse of a number on the field.

B.5 Fibonacci Sequence using Recursion

```
@zk_chip
def recur(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return recur(n - 1) + recur(n - 2)

@zk_circuit
def fibonacci(n: int, expected: int):
    if n < 20:
        # statically limit the recursion depth,
        # otherwise the result circuit would be infinite large
        assert recur(n) == expected
    else:
        assert False
```

Figure 19: ZINNIA Code for Recursive Fibonacci Sequence.

Recursion enables a function to call itself. We illustrate this feature using the Fibonacci sequence example shown in Fig. 19. While recursion is not the most efficient method for computing Fibonacci numbers, it serves as a clear example of ZINNIA's support for recursive function calls.

In the `fibonacci` function, we restrict the input to $n < 20$, providing the compiler with bounds on the input size and preventing infinite compilation from unbounded recursion. The actual recursive behavior is handled by `recur`.

Notably, prior DSLs do not support recursion, as they lack mechanisms to reason about and guarantee the termination of recursive calls.

```
@zk_circuit
def calc_sin(a: float, b: float, c: float, expected: float):
    s = a + b + c
    delta = math.sqrt(s * (s - a) * (s - b) * (s - c))
    sin_value = 2 * delta / (b * c)
    assert math.isclose(sin_value, expected)
```

Figure 20: ZINNIA Code for Sine Calculation.

B.6 Calculate Sine from Three Side Lengths

We present an additional case for non-linear numerical computation, specifically calculating the sine of an angle given three side lengths of a triangle. The code in Fig. 20 uses `math.sqrt` to compute the square root, and `math.isclose` to check if two floating-point numbers are close enough to be considered equal.

B.7 K-Means Clustering

```
@zk_circuit
def kmeans(
    data: NDArray[float, 10, 2],
    centroids: NDArray[float, 3, 2],
    classifications: NDArray[int, 10],
):
    n, d = data.shape
    classes = centroids.shape[0]
    labels = np.zeros((n, ), dtype=int)
    for _ in range(10):
        for i in range(n):
            dists = np.zeros((classes, ), dtype=float)
            for j in range(classes):
                dist = data[i] - centroids[j]
                dist = dist[0] * dist[0] + dist[1] * dist[1]
                dists[j] = dist
            labels[i] = np.argmin(dists)
        new_centroids = np.zeros((classes, d), dtype=float)
        counts = np.zeros((classes, ), dtype=float)
        for i in range(n):
            new_centroids[labels[i]] += data[i]
            counts[labels[i]] += 1.0
        for i in range(classes):
            new_centroids[i] /= counts[i]
        centroids = new_centroids
    assert labels == classifications
```

Figure 21: ZINNIA Code for K-Means Clustering.

This case study demonstrates ZINNIA's ability on multidimensional array operations and manipulations. Fig. 21 shows a K-Means clustering algorithm. The code initializes centroids, assigns points to clusters, and updates centroids based on the mean of the points in each cluster. `argmin` is used to find the assigned cluster for each point, and `zeros` initializes new arrays.

C Detailed Evaluation Report

While the main paper provides an overview of the evaluation results, we present the detailed results in this section, featuring the detailed throughput for each task across baselines.

C.1 Benchmark Suite

We present the details of the benchmark suite used in our evaluation in Table 8 and provide a justification for the selection of this suite.

Table 8: Benchmark Tasks in the evaluation (# denotes the problem ID in the respective platform).

MLAlgo	LeetCode										DS-1000					Crypt
① Neuron; ② K-Means;	Array		DP		Graph		Math		Matrix		#296	#330	#387	#453	#501	① Poseidon Hash;
③ Linear-Regression	#204	#832	#740	#1137	#3112	#997	#492	#2125	#73	#2133	#309	#360	#418	#459	#510	② Baby Jubjub ECC

Table 9: Detailed zkVMs' Proving Time (s) in RQ2 (SP1* denotes SP1's SNARK prover).

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	2.50	2.43	6.29	1.38	1.08	1.36	1.05	1.33	1.08	3.38	1.07	0.72	1.36
RISC0	14.11	28.70	57.51	3.64	3.59	1.90	1.89	3.56	3.63	1.89	1.91	3.59	3.59
SP1	15.18	28.28	41.93	8.40	7.44	7.41	7.29	6.91	7.23	7.31	7.38	7.39	7.38
SP1*	186.10	192.45	187.99	147.85	146.99	146.62	146.02	143.28	145.95	146.26	146.48	145.95	146.19
Cairo	N/A	N/A	N/A	35.95	4.87	4.90	4.89	6.69	5.53	5.54	5.96	5.36	5.61

	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon
Zinnia	1.06	1.33	1.33	1.04	1.07	1.37	1.70	1.35	1.06	1.32	1.04	1.14
RISC0	1.91	1.86	3.57	1.85	1.83	1.86	3.54	1.87	1.84	3.53	173.02	74.68
SP1	7.36	7.32	7.43	7.47	7.14	7.33	7.46	7.35	7.24	7.25	52.88	55.04
SP1*	147.61	148.08	148.13	146.54	147.47	147.41	146.88	146.60	146.77	146.52	218.34	209.78
Cairo	5.10	4.92	N/A	4.73	4.53	N/A	N/A	N/A	4.71	4.68	N/A	5.50

Table 10: Detailed zkVMs' Verifying Time (ms) in RQ2 (SP1* denotes SP1's SNARK prover).

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	8.67	9.20	17.16	5.48	3.73	4.89	4.28	4.62	4.86	10.45	4.66	4.12	4.95
RISC0	16.10	17.10	31.30	13.40	14.20	12.80	12.30	13.70	13.30	12.70	12.80	14.00	13.40
SP1	162.20	160.40	167.00	155.70	154.70	154.30	153.40	153.00	154.90	155.20	152.50	153.40	153.50
SP1*	1076.30	983.10	948.30	714.00	652.20	728.10	777.80	676.60	692.70	594.40	658.70	654.80	578.00
Cairo	N/A	N/A	N/A	1320.95	1374.22	1347.65	1376.29	1314.61	1353.91	1351.21	1346.94	1311.79	1403.96

	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon
Zinnia	4.35	5.17	5.81	4.01	5.14	6.55	6.70	5.97	4.20	4.37	3.96	4.15
RISC0	12.30	12.60	13.20	12.20	12.40	12.90	13.40	12.90	12.50	13.20	97.20	45.20
SP1	154.20	153.30	153.30	152.30	154.60	154.90	156.10	151.30	153.70	152.50	366.70	235.20
SP1*	691.20	659.70	664.80	733.80	670.20	809.80	683.70	580.60	762.00	612.60	688.70	811.30
Cairo	1319.43	1340.75	N/A	1394.45	1380.38	N/A	N/A	N/A	1315.58	1318.31	N/A	1464.30

Task Selection Criteria. Our task selection is designed to comprehensively evaluate Zinnia across a diverse set of data analytics workloads that reflect both practical and specialized use cases. The rationale behind this selection is multi-fold: First, data science tasks represent some of the most prevalent and impactful applications of data analytics in practice. To this end, we randomly selected 10 tasks from the well-established DS-1000 dataset, ensuring our evaluation captures typical data science workflows encountered by practitioners. Second, to assess general-purpose analytical and algorithmic capabilities, we chose 10 tasks from Leetcode, spanning five key categories — array, dynamic programming, graph, math, and matrix problems. These categories collectively represent a broad spectrum of algorithmic patterns and computational challenges that frequently arise in data analytics pipelines. Third, we incorporated 3 representative machine learning tasks, covering foundational algorithms that are ubiquitous in modern data-driven applications. This inclusion demonstrates Zinnia's applicability to core ML workloads, which often form the backbone of data analytics. Finally, recognizing that some users have heightened requirements for cryptographic security in data analytics, we added 2 cryptographic tasks. These examples illustrate Zinnia's support for privacy-preserving or security-critical computations, broadening its relevance to specialized scenarios.

Implementation Details. As described in the main paper, we implemented the benchmark suite in Zinnia across 6 baselines: Zinnia, Halo2, Noir, SP1, RISC0 and Cairo, respectively. ① To ensure correctness, we cross-verified that all tasks produce identical outputs across all implementations. ② All implementations share the same program structure, algorithm logic, and data flow, ensuring a fair comparison. ③ We used the same input for all implementations, ensuring that performance differences are due to the underlying system architectures and not variations in input data.

C.2 Full zkVM Comparison Result

In the main paper, we provided an overview of the comparison results across zkVM solutions. For a more detailed result, we present the concrete proving and verifying time for each task for each task across all zkVMs in Table 9 and Table 10, respectively. The results demonstrate that Zinnia consistently outperforms the other zkVMs in both proving and verifying time across all tasks. Notably, several tasks involving real number arithmetic are excluded from

the comparison with Cairo as Cairo does not support real number arithmetic.

C.3 Full Circuit Performance Result

We also provide the detailed circuit size (i.e., number of polynomial constraints) for each task on PLONK and ULTRAHONK protocols in Table 11 and Table 12, respectively. The baseline for PLONK protocol is Halo2, while the baseline for ULTRAHONK protocol is Noir.

C.4 Full Ablation Study Result

Ablation Variants. Our ablation study investigates the impact of Zinnia's symbolic execution compilation engine and different optimizers on circuit performance. We consider the following variants:

- **No Dead Code Elimination (V1):** This variant applies the same compilation engine as Zinnia and all optimizers except the dead code elimination optimizer.
- **No Common Sub-expression Elimination (V2):** This variant applies the same compilation engine as Zinnia and all optimizers except the common sub-expression elimination optimizer.
- **No Pattern Matching (V3):** This variant applies the same compilation engine as Zinnia and all optimizers except the pattern matching optimizer. Notably, pattern machine detects common patterns in the circuit and replaces them with more efficient implementations.
- **No Symbolic Execution's Pruning (V4):** This variant uses the same compilation engine as Zinnia but deliberately disables the symbolic execution's pruning capabilities.

In additional to the overview figure presented in the main paper, we provide the detailed results of the ablation study in Table 13. Each cell contains the increase in circuit size (i.e., number of polynomial constraints) compared to the no ablation variant of Zinnia.

C.5 Proof Size

Due to space constraints, we omitted the detailed proof-size results from the main paper. Here, we present them in full. Since zkVMs rely on a zk-STARKs based proof system, their proof sizes are not directly comparable to those of zk-SNARKs. Accordingly, we limit our comparison to Halo2 and Noir, running on the PLONK and ULTRAHONK protocols, respectively. The results appear in Table 14 and Table 15.

Proof sizes for zk-SNARKs are measured in bytes—the exact number of bytes transmitted over the network. Smaller proofs consume fewer network resources. As expected, proof size exhibits a positive correlation with circuit size (i.e., the number of polynomial constraints). Under the PLONK protocol, Zinnia matches or improves upon Halo2's proof size across all benchmarks. Under ULTRAHONK, Zinnia outperforms Noir on most tasks except on the three exceptions where Zinnia incurs a larger proof. This is attributable to a larger circuit size resulting from our backend-agnostic IR design, which we have already discussed in the main paper.

If-Else-Stmt

$$\frac{\Gamma,\Pi,\sigma \vdash \{e_1\} \triangleright \Gamma_1,\Pi_1,\sigma_1,\langle \mathcal{L},\mathcal{E}\rangle \quad \text{IfFrame}(\Gamma_1,\mathcal{E}),\Pi_1,\sigma_1 \vdash \{s_1\} \triangleright \Gamma_2,\Pi_2,\sigma_2 \quad \text{IfFrame}(\hat{\Gamma}_2,\neg\mathcal{E}),\Pi_2,\sigma_2 \vdash \{s_2\} \triangleright \Gamma',\Pi',\sigma'}{\Gamma,\Pi,\sigma \vdash \{\texttt{if } e_1 : s_1 \texttt{ else} : s_2\} \triangleright \hat{\Gamma}',\Pi',\sigma'}$$

Arr-Updating

$$\frac{\begin{array}{c}\Gamma,\Pi,\sigma \vdash \{e_1\} \triangleright \Gamma',\Pi',\sigma',\{\langle \mathcal{L}_1,\mathcal{E}_1\rangle,\ldots,\langle \mathcal{L}_n,\mathcal{E}_n\rangle\} \\ \Gamma',\Pi',\sigma' \vdash \{e_2\} \triangleright \Gamma'',\Pi'',\sigma'',\langle \mathcal{L}^{\text{idx}},\mathcal{E}^{\text{idx}}\rangle \\ \Gamma'',\Pi'',\sigma'' \vdash \{e_3\} \triangleright \Gamma''',\Pi''',\sigma''',\langle \mathcal{L}^{\text{new}},\mathcal{E}^{\text{new}}\rangle \\ \mathcal{E}_i^{\text{new}} = select(\mathcal{E}^{\text{idx}} == i,\mathcal{E}^{\text{new}},\mathcal{E}_i), 1 \le i \le n \\ \mathcal{L}_i^{\text{new}} = (|\sigma'''| + i), 1 \le i \le n \quad \mathcal{E}^{\text{result}} = \{\mathcal{E}_1^{\text{new}},\ldots,\mathcal{E}_n^{\text{new}}\} \\ \mathcal{L}^{\text{result}} = \{\mathcal{L}_1^{\text{new}},\ldots,\mathcal{L}_n^{\text{new}}\} \quad \mathcal{T}(\mathcal{E}^{\text{new}}) = \mathcal{T}(\mathcal{E}^{\text{idx}})\end{array}}{\Gamma,\Pi,\sigma \vdash \{e_1[e_2] \leftarrow e_3\} \triangleright \Gamma''',\Pi''',\sigma'''[\mathcal{L}_i^{\text{new}} \mapsto \mathcal{E}_i^{\text{new}}],\langle \mathcal{L}^{\text{result}},\mathcal{E}^{\text{result}}\rangle}$$

Unary-Expr

$$\frac{\begin{array}{c}\Gamma,\Pi,\sigma \vdash \{e_1\} \triangleright \Gamma_1,\Pi_1,\sigma_1,\langle \mathcal{L}_1,\mathcal{E}_1\rangle \\ \mathcal{E}_2 = \odot\,\mathcal{E}_1 \quad \mathcal{L}_2 = |\sigma_1| \quad \sigma_2 = \sigma_1[\mathcal{L}_2 \mapsto \mathcal{E}_2]\end{array}}{\Gamma,\Pi,\sigma \vdash \{\odot\, e\} \triangleright \Gamma_1,\Pi_1 \cup \{\mathcal{E}_2 \leftarrow \text{op}_\odot(\mathcal{E}_1)\},\sigma_2,\langle \mathcal{L}_2,\mathcal{E}_2\rangle}$$

Figure 22: Supplementary Operational Semantics for Zinnia's Symbolic Execution Compiler.

D Supplementary Operational Semantics

In addition to the already presented operational semantics in the main paper, we provide an extended set of rules that were omitted for brevity in Fig. 22.

E Memory Aliasing

In symbolic execution, pointers into arrays can alias, risking inconsistent reads or writes [6]. We resolve this by encoding every access and update as a per-element multiplexer. For an access at index j, we compute

$$v \;=\; \sum_{i=0}^{n-1} (i = j)\;\cdot arr[i],$$

and for an update at the same index,

$$arr[i] \;\leftarrow\; (i = j)\; v \;+\; (i \neq j)\; arr[i].$$

Because the additional multiplexers scale linearly and are bounded by the array length, this approach incurs acceptable overhead while preserving sound, alias-aware memory semantics.

Table 11: Detailed Circuit Size (No. of Polynomial Constraints) in RQ2 on PLONK Protocol.

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	354810	337120	1297398	34218	8200	14865	3652	1620	6585	638535	6174	13600	18693
Halo2	380900	354413	2256351	108756	8200	15385	4425	139781	10101	643647	80039	41440	19701
	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon	
Zinnia	399	591	118161	116	224	4436	196105	2221	720	13498	252	17	
Halo2	783	609	118161	116	224	4446	196131	2226	765	14726	258	17	

Table 12: Detailed Circuit Size (No. of Polynomial Constraints) in RQ2 on ULTRAHONK Protocol.

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	N/A	N/A	N/A	7125	1101	7966	504	3602	561	38339	1728	1681	9161
Noir	N/A	N/A	N/A	10317	1101	14647	3438	202550	3353	171899	1938	1521	32735
	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon	
Zinnia	25	3132	N/A	9	17	N/A	N/A	N/A	118	6378	17	459	
Noir	49	3670	N/A	9	17	N/A	N/A	N/A	73	3436	17	459	

Table 13: Detailed Circuit Size Increase (No. of Polynomial Constraints) in the Ablation Study in RQ3.

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
V1	2087	14396	2293	32	0	8200	25	111440	1844	8098	4954	0	0
V2	9800	20040	8	9972	0	0	12	0	716	0	8260	21760	1584
V3	2024	8280	5260	8	2400	3304	808	0	0	8	1928	0	808
V4	53474	20040	951392	64526	0	0	12	26721	956	0	58723	16320	1584
	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon	
V1	0	8	0	0	516	607	400	87	0	16	4	0	
V2	0	0	0	0	0	0	0	0	108	360	4	0	
V3	96	16	64	64	128	24	180	12	72	48	40	16	
V4	288	0	0	0	0	87	0	87	108	804	0	0	

Table 14: SNARK Proof Size (in Bytes) Comparison Results on PLONK.

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	10992	10992	33324	3844	3276	3844	3276	3844	3276	16764	3276	3276	3844
Halo2	10992	10992	54336	5676	3276	3844	3276	6384	3276	16764	5676	3276	3844
	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon	
Zinnia	3276	3844	5676	3276	3276	3844	6384	3844	3276	3844	3276	3308	
Halo2	3276	3844	5676	3276	3276	3844	6384	3844	3276	3844	3276	3308	

Table 15: SNARK Proof Size (in Bytes) Comparison Results on ULTRAHONK.

	ML-Neuron	ML-KMeans	ML-LinReg	LC-#204	LC-#832	LC-#740	LC-#1137	LC-#3112	LC-#997	LC-#492	LC-#2125	LC-#73	LC-#2133
Zinnia	N/A	N/A	N/A	7144	1116	7985	519	3621	576	38382	1745	1696	9180
Noir	N/A	N/A	N/A	10336	1116	14666	3457	202812	3372	171918	2235	1536	32754
	DS-#296	DS-#309	DS-#330	DS-#360	DS-#387	DS-#418	DS-#453	DS-#459	DS-#501	DS-#510	CRY-ECC	CRY-Poseidon	
Zinnia	40	3151	N/A	24	32	N/A	N/A	N/A	133	6397	43	2017	
Noir	64	3689	N/A	24	32	N/A	N/A	N/A	88	3459	43	2014	